

Bash Shell Scripting

Brian Marshall

Acknowledgments:

Many thanks to *Mark Gordon* (mark.gordon@sun.com) at Sun Microsystems for his input, corrections and additions and for Appendix E.

1	Introduction.....	6
1.1	Scripts	9
1.2	Creating a Shell Script	11
1.3	Executing a script.....	14
2	Comments	15
3	Statement Separators.....	16
4	Variables	17
4.1	Local and Environmental variables	17
4.2	Special Variables	19
4.3	Special Variable Notation	19
4.4	Typing variables.....	23
5	Quotes	24
5.1	Backslash	24
5.2	Single Quote.....	25
5.3	Double Quote	25
6	Assignment	27
7	Command Substitution.....	30
8	Some Basic Commands	32
8.1	echo	32
8.2	read.....	32
8.3	eval <string>	33
8.4	xargs [options] [command]	34
9	Filters	39
10	Pipes.....	40
11	Control Structures	42
11.1	Exit Status	42
11.2	Test Command.....	43
11.2.1	File tests	43
11.2.2	String tests.....	43
11.2.3	Numeric tests	43
11.2.4	Boolean operators	44
11.2.5	Caveats	44
11.3	Extended Test Command.....	45
11.4	If Statement.....	46
11.5	Case Statement.....	50
11.6	Loops.....	52
11.6.1	For Loop.....	52
11.6.2	Arithmetic For Loop	53
11.6.3	While Loops.....	53
11.6.4	Until Loops	54
11.6.5	Break statement.....	55
11.6.6	Continue statement.....	56
11.7	Select Statement.....	57
12	Command Line Arguments.....	60
13	Regular Expressions.....	62

14 Processes	64
14.1 Files	65
14.2 Input Redirection	65
14.3 Here Documents.....	65
14.4 Output Redirection.....	67
14.5 Output Appending.....	67
14.6 Opening a file with a given file descriptor.....	68
14.7 Closing a file with a given file descriptor	68
14.8 File Merging.....	69
14.9 Reading from a file	69
14.10 Writing to a file.....	71
15 Functions.....	72
15.1 bold	74
15.2 boldOff.....	74
15.3 hlt	75
15.4 hltOff.....	75
15.5 hlp	75
15.6 lower	76
15.7 upper	77
15.8 initCap.....	77
15.9 getKey	77
15.10 sws.....	78
15.11 lf	78
15.12 cf	79
15.13 vf	79
15.14 range.....	79
16 Parsing.....	81
16.1 echo	81
16.2 set	84
16.3 tr	85
16.4 cut.....	88
16.5 grep	89
16.6 awk.....	90
16.6.1 Built in variables	92
16.6.2 Example Scripts:	92
16.6.2.1 args.....	93
16.6.2.2 countBash.....	93
16.6.2.3 countWords.....	94
16.6.2.4 readFile	95
16.6.2.5 functionTest	96
16.7 sed	98
16.7.1 line addresses	100
16.7.2 substitute	101
16.7.3 delete	103
16.7.4 insert, append and change	103
16.7.5 read.....	105

16.7.6 write	106
16.7.7 Example	106
17 Sample scripts	108
17.1 xtr	108
17.2 xt	109
17.3 tree.....	111
17.4 fileManager	112
17.5 getNC3Schema	114
17.6 ftpFiles	115
setupSSH.....	117
cres	120
17.7 crech.....	123
17.8 createMakefile.....	126
18 Appendix A Bourne Shell Variables.....	133
19 Appendix B Bash Variables.....	134
20 Appendix C Command-line processing.....	140
21 Appendix D FAQs.....	141
21.1 How do I keep the read builtin from skipping leading whitespace?.....	141
21.2 How do I read from the keyboard or write to the terminal when STDIN or STDOUT is redirected?	141
21.3 How do I strip duplicate whitespace from a string?	141
21.4 Can a variable have a default value?.....	142
21.5 How can we reset \$* efficiently?	142
22 Appendix E Regular Expression Synopsis	143

Unix Shell Scripting

Learning how to effectively use the shell requires practice in order to master techniques, but also thought to understand the intricacies and underlying principles. In what follows, I recommend starting up an xterm and convincing yourself, at each step, that the operations indeed do what they advertise.

1 Introduction

The power of the Unix shells are well known. It is not uncommon to get a taste of the shell's elegance and conciseness through simple every day examples. Given the file *data*, for example, with the following contents

```
$ cat data1
```

```
a
b
x
a
c
b
```

one might be confronted with the task of removing duplicates but *without* changing the order of the elements. To this end, we can start by applying the *nl* command:

```
$ nl data
```

```
1 a
2 b
3 x
4 a
5 c
```

¹ The *cat* command, which comes from the Latin word '*catenate*' means to *chain together*. It is used to print the contents of a file to the screen, or as a techie would say, to "*standard out*". It can also be used to create files. Thus to create the *data* file in this example so that you can work through this example yourself, type:

```
cat > data
a
b
x
a
c
b
Ctrl-d
```

Here Ctrl-d means "hold the Control key down while pressing the 'd' key". Ctrl-d is the end-of-file character in Unix and lets the *cat* command know it's done processing.

```
6 b
```

which simply prepends a line number to each line of the file. The resulting output can then be sorted by the second column:

```
$ nl data | sort -k 2
```

```
1 a
4 a
2 b
6 b
5 c
3 x
```

from which duplicates in the sorted second column can be removed:

```
$ nl data | sort -k 2 | uniq -f 1
```

```
1 a
2 b
5 c
3 x
```

uniq operates on sorted data only, and in this example, by using the `-f 1` (that is “one” and not the letter ‘elle’) option, we can specify for which column, data should be unique. Duplicates have thus been removed successfully, but the relative ordering is not the same as in the file. For this, the lines need to be sorted – in numerical order (and that is why we pass the command-line switch “-n” to the *sort* command - on the first column:

```
$ nl data | sort -k 2 | uniq -f 1 | sort -n
```

```
1 a
2 b
3 x
5 c
```

and finally the first column needs to be removed:

```
$ nl data | sort +1 | uniq -f 1 | sort -n | awk '{print $2}'
```

```
a
b
x
c
```

Here we use the *awk* command to print out just the second column. This command will be examined in detail further on in this tutorial.

It is often mentioned in the literature that *Unix* is the first *Component Model*. This simple example, which combines five commands (components), glued together by the pipe symbol, '|', is testimony to that.

What makes the shell even more powerful is the fact that the above sequence can be stored in a file, which can be made executable, creating, in effect, a brand new command – a script.

1.1 Scripts

A *shell script* is a text file that contains commands to be executed by the shell. For example, a script to list the newest file in the current directory could be as follows (where the hash character '#' starts a *comment* and thus doesn't influence the workings of the script:

```
$ cat lf
```

```
# list the newest file in the current directory. By default, tail lists
# the last ten lines of output. Here, by specifying -n -1 (minus one), we
# list the last line of output.
ls -ltr2 | tail -n -13
```

To execute the script in the bash shell, issue:

```
$ bash lf
```

But the script can be invoked on its own if you make the file executable by typing:

```
$ chmod +x lf
$ lf
```

```
<lists the newest file in the current directory>
```

The *lf* script is executed using the current shell. Since there are no shell specific commands in this script, it will run under any shell.

To determine which shell you are running, try executing:

```
$ echo $0
```

or

```
$ echo $SHELL
```

² The Unix *ls* command lists files in the current directory, by default sorted by file name. Here the -t switch means 'sort by modified time' instead, and -t -r means 'sort in reverse time order' and thus lists the newest files last. The two switches -t -r can be combine as -tr. Finally the -1 (minus one) means '*list the files in one column*'.

³ The Unix *tail* command with no command-line switch prints the last 10 lines of a file. But to print the last 5 lines, say, of the */etc/passwd* file, issue:

```
tail -n 5 /etc/passwd
```

or equivalently:

```
tail -n -5 /etc/passwd
```

To print the contents of that file starting line 5, issue:

```
tail -n +5 /etc/passwd
```

instead.

or type the following to get a full (-f) list the processes running in your shell:

```
$ ps -f
```

The former prints, for the Bourne family of shells (not the C-Shell), the name of the running shell. Thus:

```
echo $0
```

```
bash
```

```
$ csh      # now running the C-Shell
$ lf      # still lists the newest file
```

If the script had had bash specific constructs, however, this wouldn't work:

```
$ cat lf
```

```
# list the newest n files in the current directory.
# ${1:-1} is the value of the first command line argument
# if specified, otherwise 1.
# This works in bash, bourne, korn, but not csh
ls -ltr | tail -${1:-1}
```

Now the command:

```
$ lf 5
```

will run and list the newest 5 files in the current directory when run from within bash, but:

```
$ csh      # now running the C-Shell
$ lf 5     # variable syntax (i.e. csh is confused since it doesn't
           # understand the syntax ${1:-1})
```

Yet:

```
$ bash lf 5  # lists the newest 5 files
```

works from within csh, because we are explicitly running *lf* using bash.

Specifying one of the following as the first line in a script:

```
#!/usr/bin/bash4      # full path to bash
```

⁴ From the command line, write:
type bash

```
#!/usr/bin/env bash5
```

lets the shell know which shell (possibly different than the one we are running in) should be invoked to execute the commands in the script. ‘#!’ is called ‘she-bang’ (hash-bang). When an executable file is invoked, the kernel looks into the file. If the first two characters of the file are “#!”, it invokes the shell (or command) specified on that line and feeds all following lines to that shell.

1.2 Creating a Shell Script

Thus, to create a shell script:

1. Use the she-bang notation to determine which shell should be invoked. This should be the first line of the script.
2. Write the script and save the file.
3. Make the file executable: `chmod6 u+x fileName` or `chmod ug+x fileName`

To alleviate the creation of bash scripts, we present the following script, named `cres` (create script), that automates the drudgery of creating a script. It is well commented, but we will cover its details in the course of these notes.

to find out where the bash command is. In my case, that command returns `/usr/bin/bash`. Should yours return `/bin/bash`, use that instead.

⁵ The advantage of this syntax is that the `bash` command just needs to be somewhere in the `PATH`.

⁶ The `chmod` command “changes the mod” of a file. `+x` makes a file executable for anyone. `u+x` makes it executable only for the owner of the file and `ug+x` makes it executable for the owner and anyone in its group.

```

#!/bin/bash
#=====
# File      : cres
# Description : Create script.  Prompt for a filename if one isn't specified
#              on the command line.  Check if the file exists.  Add the basic
#              lines to the script and then make it executable
#
# Revision History:
#
#   10 Aug 1999 - bmarshal
#       Initial Version.
#=====
#-----
# Get the filename either from the command line if specified or after
# prompting the user for it.  Here's how this works: If a command line arg
# is specified, it is stored in $1.  If this is defined, assign it to
# fileName.  Otherwise, prompt the user for a fileName (read -p "prompt"),
# read the user's input into resp and return it (by echo'ing it).
#-----
fileName=${1:-$(read -p "Enter fileName: " resp; echo $resp)}

#-----
# Loop until fileName is not empty (-n) and (-a or &&) a file by that name
# does not exist (! -e)
#-----
until [[ -n "$fileName" && ! -e "$fileName" ]]
do
    #-----
    # If fileName is empty (-z for zero) the re-prompt the user and start
    # at the top of the loop again (continue)
    #-----
    if [[ -z "$fileName" ]]; then
        read -p "Enter fileName: " fileName
        continue
    fi
    #-----
    # If we get here, the file exists, so ask the user if it should be
    # overwritten.  Here 'echo -n' specifies 'no new line' and
    # 'read -n 1' reads a single character from stdin into response.
    # If the user types 'y' or 'Y', break out of the until loop...
    # Note: the [[ ]] syntax is Korn or Bash syntax, whereas [] is Bourne...
    #-----
    echo -n "$fileName exists. Overwrite (y/n)? "
    if [[ $(read -n 1 response; echo $response) == [yY] ]] ; then
        break
    fi
    #-----
    # ... otherwise prompt the user for another file name
    #-----
    echo
    read -p "Enter fileName: " fileName
done

#-----
# Get the user's name, which is field 5 of the /etc/passwd (GCOS). Do

```

```

# this by grep'ing for the user's id (id -un on Linux) or login name (LOGNAME)
# in the /etc/passwd file and then cutting out the fifth colon delimited field
# Linux: userName=$(grep $(id -un) /etc/passwd | cut -d: -f5)
#-----
userName=$(grep "^$LOGNAME:" /etc/passwd | cut -d: -f5)
read -p "Comment []: " comment

#-----
# An example of a 'here' document. It says cat (write) to $fileName every
# thing after the marker EOF and before that same marker. The '-' (<<-)
# says that leading tabs are allowed and will be stripped before writing
# the here document to the file
#-----
cat > $fileName <<-EOF
    #! $prgm
    #@(#) $comment
    #=====
    # File           : $fileName
    # Description    :
    #
    # Revision History:
    #
    #   $(date +%d %h %Y) - $userName
    #   Initial Version.
    #=====

EOF

#-----
# Make the file executable (or chmod u+rwx,go-w,go+rx $fileName)
#-----
chmod 0755 $fileName

#-----
# 'vi +11 fileName' means vi fileName and place the cursor on line 11.
# We want to place the cursor on the last line of fileName so that the user
# can immediately start editing the script. Therefore first calculate the
# number of lines in the file: line=$(wc -l $fileName | awk '{print $1}').
# Then overlay vi : exec vi +$line $fileName. These two steps can be
# combined:
#-----
exec $EDITOR +$(wc -l $fileName | awk '{print $1}') $fileName

```

1.3 Executing a script

A script is started in one of two ways.

If it is started by simply typing its name at the command prompt, the shell itself is *forked*, creating a subshell and the script code overlays the code of the subshell using the *exec* system call. Thus any changes a script makes to its environment – including changes to variables, both local and environmental, changes to current working directory, and so on, are not and cannot be noticeable to the parent shell. If the child needs to communicate with the parent, some form of IPC (interprocess communication) must be used.

Otherwise it can be started using the `.` builtin. Running it this way:

```
$ . myScript
```

executes the script in the *current environment*. Thus any changes made by the script run in this way *are*, of course, noticeable to the shell in which it was invoked. Executing a script in this way is called *sourcing* the script. The initialization files (`.profile` and `ilk`) are invoked this way.

2 Comments

A comment is initiated by the character ‘#’ and extends for the rest of the line. There are no multi-line comments in the shell. If the first two characters of the script are ‘#!’ (the she-bang), the kernel knows to invoke the shell or command specified on the line and feed the rest of the file to that shell (or command).

3 Statement Separators

Each command of a script usually appears on a line by itself. Multiple commands can, however, be placed on the same line if they are separated by a statement separator. These are:

- ; # semicolon
- | # pipe
- & # run in the background
- || # ORed execution
- && # ANDed execution

4 Variables

Variables are at the heart of every programming and scripting language. They appear in arithmetic operations and manipulation of quantities, string parsing, and are indispensable for working in the abstract with symbols - tokens that represent something else. A variable is nothing more than a location or set of locations in computer memory holding an item of data.

A variable name must match the regular expression `[A-Z_a-z]+[0-9A-Z_a-z]*`. Thus a variable name must consist of characters in the range `[0-9A-Z_a-z]` and must start with a non-digit.

If *name* is a variable, then *\$name* is its value. Braces can be used to delimit the name of a variable:

```
$ group=staff
$ echo "$group = $group"

$group = staff

$ echo "There are two $groups"

There are two
```

whereas

```
$ echo "There are two ${group}s"

There are two staffs
```

Without the braces, the shell looks for a variable named *groups* and prints its value. With the braces, the shell knows to *interpolate* the value of the *group* variable.

There are two kinds of variables: *local* and *environmental*.

4.1 Local and Environmental variables

Variables defined in a shell are normally not available in a subshell.

```
$ project=nc3
$ echo $project
```

```
nc3
```

```
$ bash          # start up a subshell
$ echo $project # displays nothing
$ exit          # exit subshell
```

displays nothing. More:

```
$ cat echoProject

#! /usr/bin/bash

echo $project

$ ./echoProject
```

again displays nothing. The variable *project* isn't available from within a script since the script, when executed, actually runs in a subshell. But:

```
$ export project # project is now an environmental variable i.e. available ...
$ echoProject    # ... in any subshell
```

```
nc3
```

shows that once a variable is exported – making it an *environmental* variable – it is available in a subshell, hence in all scripts started from the current shell.

In the bash shell, once a variable is exported, it remains exported. There is no need to re-export it if its value is changed at some point, as was the case with the Bourne shell (*sh*).

To display all environmental variables, use the *env* or *export* commands:

```
$ export

... displays all environmental variables
```

or

```
$ env

... displays all environmental variables
```

On the other hand, to display all variables:

```
$ set

... displays all variables
```

Once a variable, environmental or not, is defined, it can be undefined using the *unset* command:

```
$ project=nc3
$ set | grep project

project=nc3

$ unset project
$ set | grep project

# displays nothing
```

4.2 Special Variables

The following list of special variables are well worth remembering:

\$0	# the name of the shell
\$1 – 9	# the first 9 command line arguments
\$\$	# the PID of the current process
\$-	# the shell options
#!	# the PID of the last child that died
\$?	# the exit status of the last run command
\$#	# the number of command line arguments
\$*	# the string consisting of all command line arguments
\$@	# the string consisting of all command line arguments

4.3 Special Variable Notation

In this section, we introduce special notation used in conjunction with variable name expansion.

In each of the cases below, word is subject to tilde expansion, parameter expansion, command substitution, and arithmetic expansion. When not performing substring expansion, bash tests for a parameter that is unset or null; omitting the colon results in a test only for a parameter that is unset.

`${parameter:-word}`

Use Default Values. If parameter is unset or null, the expansion of word is substituted. Otherwise, the value of parameter is substituted.

A few examples are in order:

```
$ x=3
$ echo ${x:-5}
```

3

```
$ unset x
$ echo ${x:-5}
```

5

This syntax is often used to specify default values for shell scripts or functions.

```
#!/bin/bash
#=====
# File : getFileInfo
#=====
#-----
# fileName is either the first command line argument, if it
# exists. Otherwise, using command substitution, the user is
# prompted for a file name.
#-----
fileName=${1:-$(read -p "Enter fileName: " resp; echo $resp)}
count=${2:-10}
echo "Operating on $fileName $count times"
```

`${parameter:=word}`

Assign Default Values. If parameter is unset or null, the expansion of word is assigned to parameter. The value of parameter is then substituted. Positional parameters and special parameters may not be assigned to in this way.

A few examples are in order:

```
$ unset x
$ echo ${x:=defaultValue}

defaultValue

$ echo $x

defaultValue
```

This syntax is often used to set a variable to a default value if it is not already set. Instead of the lengthy

```
$ if [[ -z $x ]] ; then
    x=defaultValue
fi
```

we can use the following idiom:

```
$ : ${x:=defaultValue}
```

The leading `:` is the NOOP builtin.

`${parameter:?word}`

Display Error if Null or Unset. If `parameter` is `null` or `unset`, the expansion of `word` (or a message to that effect if `word` is not present) is written to the standard error and the shell, if it is not interactive, exits. Otherwise, the value of `parameter` is substituted.

Use this notation if there is no sense in continuing if a variable is not set:

```
$ fileName=${1:?}"Usage: $0 <file name>"}
```

will exit a running script if it was called without a command line argument.

`${parameter:+word}`

Use Alternate Value. If `parameter` is `null` or `unset`, nothing is substituted, otherwise the expansion of `word` is substituted.

Use this notation if an alternate value is desired in the case that *parameter* is set:

```
$ args=${1:+*}
```

will assign to *args* all command line arguments if at least one was specified.

`${parameter:offset}`

`${parameter:offset:length}`

Substring Expansion. Expands to up to `length` characters of `parameter` starting at the character specified by `offset`. If `length` is omitted, expands to the substring of `parameter` starting at the character specified by `offset`. `length` and `offset` are arithmetic expressions. `length` must evaluate to a number greater than or equal to zero. If `offset` evaluates to a number less than zero, the value is used as an offset from the end of the value of `parameter`. If `parameter` is `@`, the result is `length` positional parameters beginning at `offset`. If `parameter` is an array name indexed by `@` or `*`, the result is the `length` members of the array beginning with `${parameter[offset]}`. Substring indexing is zero-based unless the positional parameters are used, in which case the indexing starts at 1.

`${!prefix*}`

Expands to the names of variables whose names begin with `prefix`, separated by the first character of the IFS special variable.

`${#parameter}`

The length in characters of the value of `parameter` is substituted. If `parameter` is `*` or `@`, the value substituted is the number of positional parameters. If `parameter` is an array name subscripted by `*` or `@`, the value substituted is the number of elements in the array.

```
$ files=(ls)
$ echo ${files}
```

```
c/ c++/ class/ doxygen/ java/ make.tar.gz
makefiles/ mysql-3.23.51/ perl/ sed/ shell/
sun/
```

```
$ echo ${#files}
```

91

```
${parameter#word}
```

```
${parameter##word}
```

The word is expanded to produce a pattern just as in pathname expansion. If the pattern matches the beginning of the value of parameter, then the result of the expansion is the expanded value of parameter with the shortest matching pattern (the ``#`` case) or the longest matching pattern (the ``##`` case) deleted. If parameter is `@` or `*`, the pattern removal operation is applied to each positional parameter in turn, and the expansion is the resultant list. If parameter is an array variable subscripted with `@` or `*`, the pattern removal operation is applied to each member of the array in turn, and the expansion is the resultant list.

Use this notation to left truncate a pattern. With one `#` it truncates modestly:

```
$ file=info2nc3.cpp.ORIG
$ echo ${file#*.}
```

```
cpp.ORIG
```

With two `#` it truncates greedily:

```
$ file=info2nc3.cpp.ORIG
$ echo ${file##*.}
```

```
ORIG
```

Again, give a full path in `$file`:

```
echo ${file##*/}
```

gives the file's *basename*

```
${parameter%word}
```

```
${parameter%%word}
```

The word is expanded to produce a pattern just as in pathname expansion. If the pattern matches a trailing portion of the expanded value of parameter, then the result of the expansion is the expanded value of parameter with the shortest matching pattern (the ``%`` case) or the longest matching pattern (the ``%%`` case) deleted. If parameter is `@` or `*`, the pattern removal operation is applied to each positional parameter in turn, and the expansion is the resultant list. If parameter is an array variable subscripted with `@` or `*`, the pattern removal operation is applied to each member of the array in turn, and the expansion is the resultant list.

Use this notation to right truncate a pattern

```
$ file=info2nc3.cpp
$ echo ${file%.cpp}.o
```

```
info2nc3.o
```

Again, give a full path in \$file:

```
echo ${PATH%/*}
```

gives the file's *dirname*.

`${parameter/pattern/string}`

`${parameter//pattern/string}`

The pattern is expanded to produce a pattern just as in `pathname` expansion. Parameter is expanded and the longest match of pattern against its value is replaced with string. In the first form, only the first match is replaced. The second form causes all matches of pattern to be replaced with string. If pattern begins with #, it must match at the beginning of the expanded value of parameter. If pattern begins with %, it must match at the end of the expanded value of parameter. If string is null, matches of pattern are deleted and the / following pattern may be omitted. If parameter is @ or *, the substitution operation is applied to each positional parameter in turn, and the expansion is the resultant list. If parameter is an array variable subscripted with @ or *, the substitution operation is applied to each member of the array in turn, and the expansion is the resultant list.

4.4 Typing variables

The builtin *typeset* or its synonym *declare* can be used to *type* bash variables. Variables typed within a function body are local the function. In a nutshell, the options to *typeset* are:

- `typeset -i num` # defines num as a integer (arithmetic is now quicker)
- `typeset -l str` # defines str as a *lowercase* string
- `typeset -u str` # defines str as an *uppercase* string
- `typeset -r var` # defines var as a readonly variable

5 Quotes

The shell provides three ways of quoting to protect text from interpretation by the shell:

- the backslash character
- single quotes
- double quotes

When quoted text is processed by the shell, the quotes are removed and the unquoted text is passed to the command. Thus in:

```
$ echo \*
```

the backslash is removed from the command line argument and the single character ‘*’ is passed to the *echo* command. The intent of the backquote is to keep the shell from interpreting the *wildcard*.

Again, for the command:

```
$ myCommand 'one two' 'three four'
```

the quotes are used to group words into command line arguments, with the result that this command has two command line arguments, ‘one two’ and ‘three four’. Here the single (or double) quotes serve to ‘escape’ the special meaning of white space as a token delimiter. The quotes are removed and the unquoted arguments are passed to *myCommand*.

5.1 Backslash

The backslash ‘quotes’ or ‘escapes’ the single character that follows it. It directs the shell to ignore any special meaning attached to that character. Thus:

```
$ echo \* \$
```

gives

```
* $
```

and

```
$ echo hello \  
world
```

```
hello world
```

shows that the newline character is not interpreted as the end of the echo command, but rather, that it is part of the string fed to the the command.itself. In effect it has been escaped. Note that the *echo* command removes tabs and newlines and replaces them with a single whitespace.

The act of placing a backslash in front of a character is called *backquoting* the character.

5.2 Single Quote

The single quote guards the text from *variable name expansion* and *command substitution*. Special characters are not interpreted. Thus the command:

```
$ name=Giulia
$ echo 'Her name is $name. Today is $(date)'
```

prints

```
Her name is $name. Today is $(date)
```

and

```
$ echo '* ? "'
```

prints

```
* ? "
```

A string enclosed withing single quotes is said to be *strongly quoted*.

5.3 Double Quote

Most special characters within double quotes are not interpreted, and only \$, ` and \ retain their special meaning, but *variable name expansion* and *command substitution* do occur. Thus the command:

```
$ name=Giulia
$ echo "Her name is $name. Today is $(date +%d %h %Y)'"
```

prints

```
Her name is Giulia. Today is 29 Jan 2003
```

and

```
$ echo "'* ? '"
```

prints

* ? \

A string enclosed withing double quotes is said to be *weakly quoted*.

6 Assignment

A value can be assigned to a variable using the syntax:

```
$ varName=value [command]
```

Note that there cannot be any spaces on either side of the '=' sign.
So the following results are not surprising:

```
$ x=2 y=6
$ echo "x =$x and y = $y"

x = 2 and y = 6
```

However, examine the following script:

```
$ cat echoVars
```

```
#!/bin/bash
#=====
# File           : echoVars
# Description    :
#
# Revision History:
#
#   28 Jan 2003 - bmarshal
#   Initial Version.
#=====
#-----
# Note: The variable is not defined in this script
#-----
echo "x = $x and y = $y"
```

```
$ x=3 y=7 echoVars
```

```
x = 3 and y = 7
```

Here the assignments occur and subsequently the script `echoVars` is invoked. Using this syntax, the variables that prefix the script are available from within the script even though they are not defined within the script itself.

But, when assignments are used in conjunction with execution, as in this example, the assignment is only valid for the duration of the script's execution.

Witness:

```
$ echo "x=$x and y=$y"
```

```
x = 2 and y = 6
```

as initially set.

This, incidentally, explains why there cannot be any spaces on either side of the '=' sign: a space after an assignment denotes the start of a command, during the execution of which, the variables are set.

On the other hand,

```
$ x=3 y=7; echoVars
```

```
x = and y =
```

gives a different answer. Since ';' is a statement separator, the assignments occur before the script is invoked. Since these variables aren't exported, their values are not available from within *echoVars*.

If the variables are exported, then their values are available in all subshells:

```
$ x=3 y=7
```

```
$ export x y # make variables x and y available to subshells
```

```
$ echoVars
```

```
x = 3 and y = 7
```

The following example uses the TZ environmental variable, whose value is the TimeZone the date and time are written in.

```
$ echo $TZ
```

```
US/Mountain
```

```
$ date
```

```
Wed Jan 29 11:13:33 MST 2003
```

But

```
$ TZ=EST date
```

gives the current Eastern Standard Time.

```
Wed Jan 29 13:13:33 MST 2003
```

Finally,

```
$ echo $TZ
```

```
US/Mountain
```

shows that the TimeZone variable, TZ, was changed only for the duration of the *date* command.

7 Command Substitution

The back quote is used to delimit a command that is to be replaced with its output.

Thus:

```
$ date
```

```
Tue Jan 28 09:47:10 MST 2003
```

```
$ today=`date`
```

```
$ echo "today is $today"
```

```
today is Tue Jan 28 09:47:10 MST 2003
```

Note how the *date* command is executed and its output substituted in place of the command itself. Also note that *command substitution* does occur within double quotes, but not single quotes.

A newer and better way to perform *command substitution* is using the `$(command)` notation. The above example condensed a bit is:

```
$ echo "today is $(date)"
```

```
today is Tue Jan 28 09:47:10 MST 2003
```

which produces the same output as above.

The advantage of the new form of command substitution is that the *start* and *end* delimiters of *command substitution* are different, which allows easy nesting. Thus assume there is a function, *upper*, that takes a string and returns it uppercased. Then the following statement

```
$ id=$(upper $(read -p "Enter ID: "; echo $REPLY))
```

prompts the user for an ID, uppercases it and assigns it to `id`.

The same sequence of commands can be issued using back quotes, but it is significantly harder to read:

```
$ id=`upper `read -p "Enter ID: "; echo $REPLY``
```

The backslashes, which ‘escape’ the backquotes are necessary. Without the first backslash, for example, the second backquote would end the first.

The following script uses *command substitution* to edit any executable file in the current PATH.

```
#!/bin/bash
# @(#) vi which fileName
#=====
# File           : vw
# Description    :
#
# Revision History:
#
#   17 Dec 2002 - marshallb
#       Initial Version.
#=====
#-----
# $1 is the first command line argument.  The value of $0 is the
# name of the script (vw).  The value of ${1:? "Usage: $0 <fileName>"}
# is the first command line arg if it is defined.  If it is NOT
# defined, this expression guarantees that the usage
# statement is printed to STDERR and the script is exited.
# The which command walks the $PATH variable in search of an
# executable.  The output of the which command is fed to vim
# (vi improved).
#-----
vim $(which ${1:? "Usage: $0 <fileName>"})
```

Thus from any directory, one can issue:

```
$ vw creb
```

to edit the script *creb*. The *which* command walks the \$PATH variable and returns a path to *creb*, which is then passed on to *vim*.

8 Some Basic Commands

In the following document, we will often encounter the following commands or command switches of commands that are lesser known.

8.1 *echo*

This well known command prints text to STDOUT.

```
$ echo "Hello\nWorld"

Hello\nWorld
```

But notice that the characters “\n” aren’t interpreted as a newline character. The `-e` option fixes that:

```
$ echo -e "Hello\nWorld"

Hello
World
```

By default, the *echo* command prints a newline character after displaying what is passed to it. This can be avoided by using the `-n` option:

```
$ echo -n "Enter your ID: "

Enter your ID: _
```

8.2 *read*

A user can read from STDIN by using the *read* command. It takes a number of variables as arguments and populates them with the tokens read from STDIN. The last argument to the *read* command captures *all* the remaining tokens.

The token delimiters are stored in the environmental variable IFS (Internal Field Separator), which by default consists of a single space and a tab.

Thus:

```
$ IFS=: read uid passwd uid remainder < /etc/passwd
```

reads the first colon-delimited token from STDIN, which *is* `/etc/passwd` because of the redirection, into *uid*, the second colon-delimited token into *passwd*, the third colon-

delimited token into *uid* and *all* remaining tokens into *remainder*. In this neat idiom, IFS is set to ‘:’ only for the duration of the *read* command.

To prompt the user before reading, use the *-p* option:

```
$ read -p "Enter your ID: " id
Enter your ID: 1234

$ echo "${LOGNAME}'s ID is $id"
marshall's ID is 1234
```

This option is useful is since the prompt is guaranteed to be flushed to STDOUT before the *read* command blocks. Simply *echo*’ing the prompt before *read*’ing doesn’t insure that the prompt will be displayed before blocking on the *read* since STDOUT is buffered.

Another useful option of the *read* command is the *-n* option. *read -n <number>* forces *read* to return after reading *<number>* characters. Thus in the following example:

```
$ read -n 1 -p "Press 'y' or 'Y' to continue" contOrEnd
$ if [[ $contOrEnd != [yY] ]]
then
    return
fi
```

the user is prompted whether to continue or not. Processing continues after reading one single character from the keyboard, without the need of hitting the <Enter> key. Here the expression *[yY]* is the regular expression (see below) that matches either a ‘y’ or a ‘Y’.

The last useful *read* option presented (but see the man pages) is the *timed* option *-t*:

```
$ read -p "Enter group : " -t 5 group
$ : ${group:=staff}
```

prompts the user to enter her group. If she doesn’t enter a response within 5 seconds, the *read* returns and if the variable, *group*, isn’t set in the preceding code somewhere, it is assigned the value ‘staff’.

8.3 *eval <string>*

The *eval* builtin takes a string as a parameter and executes it. This is a very useful command for dynamic programming. As a simple example:

```
$ for i in $(range 5)
```

```
do
    eval "myVar_$i=$i"
done
```

creates five the variables `myVar_1` ... `myVar_5` and assigns them the values 1 ... 5 respectively.

The following use of *eval* allows a developer to change directory from a source directory in her HOME directory to the same directory under `$ITEST`.

```
$ alias sabdir='eval cd $ITEST/src/$(basename $(pwd))'
```

Note that the single quotes guarantee that variable name expansion and command substitution do not occur when this alias is defined.

Now from the command line :

```
$ pwd
/home/marshall/nc3migr/src/info2nc3

$ sabdir
$ pwd
/images/sablime-build/NC3/nc3migr/itest/src/info2nc3
```

changes directory to the equivalent directory underneath `$ITEST` and

```
$ cd -      # cd back to the previous directory
$ pwd
/home/marshall/nc3migr/src/info2nc3
```

switches back to the previous directory. The key to above discussion is the late-binding effect that is obtained by using *eval* with the command string `"cd $ITEST/src/$(basename $(pwd))"`

For a more elaborate example, see the filemanager script below.

8.4 *xargs* [options] [command]

xargs is a powerful *Unix* command which is called as follows:

```
xargs [options] [command]
```

It invokes *command* and passes to it the argument list it reads from STDIN. In effect, *xargs* creates an argument list for *command* from standard input.

If *command* is not specified, it defaults to the *Unix echo* command. Thus:

```
$ xargs
      a b          # enter data from keyboard
      c d
      Ctrl-D
```

produces

```
a b c d
```

because *xargs* invokes the *echo* command— which as you will recall consumes whitespace with the result that what was input on two separate lines is output on one - and passes to it the arguments it reads from STDIN.

xargs is typically used in a pipeline. Thus the two commands:

```
$ cmd1 $( cmd2 )
$ cmd2 | xargs cmd1
```

are *conceptually* equivalent. In the first case, *cmd2* is executed using *command substitution* and its output becomes the command line arguments for *cmd1*. In the second case, the output of *cmd2* is written to STDOUT, which because of the *pipe* is input to *xargs*. This command takes that input and invokes *cmd1* with that input as command line arguments.

So, if

```
$ ls *.cpp
      a.cpp
      b.cpp
      c.cpp
```

then

```
$ wc -l $(ls *.cpp)
```

expands to

```
$ wc -l a.cpp b.cpp c.cpp
      303  a.cpp
      482  b.cpp
```

```
174 c.cpp
```

Similarly, for the pipeline:

```
$ ls *.cpp | xargs wc -l
```

```
303 a.cpp
482 b.cpp
174 c.cpp
```

the *ls* command writes *a.cpp*, *b.cpp* and *c.cpp* to STDOUT, *xargs* invokes *wc -l* and reads the command line arguments from STDIN, which is the STDOUT of the previous command, and the results are the same.

xargs is advantageous when the number of command line arguments is very large, since there is a limit to the number of arguments allowed for a single command line. *xargs* will bunch the arguments into smaller groups and execute *command* separately for each of them.

Thus if one wanted to find all C++ files that contain the word TODO, the following:

```
$ grep -i todo $(find / -name "*.cpp")
```

might fail with a message akin to:

```
grep: too many arguments
```

but the command

```
$ find / -name "*.cpp" | xargs grep -i todo
```

will succeed, allowing *grep* to process more arguments than it could normally handle.

How many arguments can be passed to a command on the command line? To try the following experiment:

```
$ range 3000 | tr ' '\n' > bigFile
```

will create the file *bigFile* consisting of the numbers in the range 1 .. 3000, each on its own line. Thus

```
$ cat bigFile | wc
```

results in

```
3000 3000 13893 # number of lines, words, characters in bigFile
```

Now if we pipe *bigFile* through *xargs* with no arguments, *xargs* will invoke the *echo* command and pass it as large a bundle of arguments as *echo* is willing to accept:

```
$ cat bigFile | xargs | wc  
12 3000 13893
```

This shows that the *echo* command couldn't handle all 3000 arguments at once, but rather was invoked 12 separate times and

```
$ cat bigFile | xargs | while read line  
do  
    echo $line | wc -w  
done  
254  
254  
254  
254  
254  
254  
254  
254  
254  
254  
254  
206
```

shows that for each invocation, *echo* was able to handle 254 tokens, with the exception of the last line for which there were only 206.

The `-n <num>` option of *xargs* executes a *command* with up to *num* arguments. Thus with the `-n1` option

```
$ ls | xargs -n1  
a.cpp  
b.cpp  
c.cpp  
d.cpp
```

xargs only passes one argument at a time to the *echo* command, whereas using the `-n3` option

```
$ ls | xargs -n3
```

```
a.cpp b.cpp c.cpp  
d.cpp
```

xargs passes up to three arguments at a time to *echo*.

Expanding on this idea, the contents of a file with one word per line can be displayed as follows:

```
$ cat fileName | xargs -n1
```

The `-l <num>` option of *xargs* executes a command every *num* lines of input. For example,

```
$ cat myFile
```

```
line 1  
line 2  
line 3  
line 4
```

The file *myFile* has four lines.

```
$ cat myFile | xargs
```

```
line 1 line 2 line 3 line 4
```

With no options *xargs* bundles lines of input into the longest arguments list that the shell can process. In this case all lines are bundled into one argument list. Use `-l2` to bundle every two lines of input together.

```
$ cat myFile | xargs -l2
```

```
line 1 line 2  
line 3 line 4
```

9 Filters

A filter is a *Unix* command that *can* accept input from STDIN and prints output to STDOUT. There are many filters, some well known of which are *sort*, *tr*, *sed*, *awk*, *grep* and *uniq*. The concept of *filter* is crucial in *Unix* because these commands can be used in pipelines.

Many filters can accept command line arguments. For such a filter, it is assumed that these are file names it is to operate on. If on the other hand no parameters are passed to such a filter, it operates on STDIN. In other words, for filters that *can* operate on other files, STDIN is the default file it operates on if *no* other files are specified.

A few examples will help clarify these points.

```
$ sort file1 file2 file3
```

produces sorted output of the combined files file1, file2, file3. Since *sort* was passed parameters, it assumes they are file names operates on these.

Equivalent results are obtained by issuing:

```
$ cat file1 file2 file3 | sort
```

although this sequence of commands is not as efficient as the former. Here the three files are (con)catenated and the results are fed to *sort*. The latter command has no command-line arguments, hence no files explicitly mentioned to operate on. It therefore operates on STDIN – its default file. But because of the pipe, the STDOUT of the *cat* command *is* the STDIN of the *sort* command. So, in effect, *sort* operates on the files, albeit indirectly.

10 Pipes

Recall that a pipe, '|', joins two commands. When

```
$ ps -ef | awk '{print $2}'
```

is issued, the two commands – `ps -ef` and `awk '{print $2}'` – are executed in two separate children processes of the shell that command was issued from. But before these children are *forked*, the output of the former shell is joined to the input of the latter. The input of the latter therefore *is* the output of the former. Now since `awk` is a filter and isn't operating explicitly on a file, it operates on its default file – `STDIN`, which because of the pipe *is* the `STDOUT` of the `ps` command.

In the early days of *Unix* the above command would have taken three steps:

```
$ ps -ef >/tmp/ps.out.$$ ; awk '{print $2}' /tmp/ps.out.$$ ; rm /tmp/ps.out.$$
```

in which the output of the `ps` command is written to a temporary file, on which `awk` operates. Note the often used *Unix* idiom of creating a temporary file name by appending the current process' PID.

Observe that not all filters take command line arguments. Such filters operate on only one file: `STDIN`. *uniq* is such example.

In closing this section, the following example shows that all the scripts executing in a pipeline are children of the shell the pipeline is started from.

```
#!/usr/bin/bash
# @(#)
#=====
# File           : startPipeTest
# Description    :
#
# Revision History:
#
#   30 Jan 2003 - bmarshal
#       Initial Version.
#=====
#-----
# This script kicks off the pipeTest. Recall that $$ is the process
# id of the currently running process, $PPID is the PID of its parent.
#-----
echo "Starting $0 with PID = $$ from shell with PID = $PPID" > /dev/tty
echo "$$: $PPID" | pipeTest | pipeTest | pipeTest
echo "$$: $PPID"
```

The `startPipeTest` script kicks off the test. It writes the string “`$$: $PPID`” down a pipe.

\$cat pipeTest

```

#!/usr/bin/bash
# @(#)
#=====
# File           : pipeTest
# Description    : This script shows that all processes in a
#                  pipeline are children of the shell the
#                  pipeline is invoked from.
#
# Revision History:
#
#   30 Jan 2003 - bmarshal
#   Initial Version.
#=====
#-----
# When this process starts, print a diagnostic message to the
# terminal. This file, identified by /dev/tty, is usually the same
# as STDOUT. But if the output of this script is redirected ('>'
# or '|'), STDOUT and /dev/tty differ.
# Here $$ is the special variable that stores the current process'
# PID, and $PPID is the parent process' PID
#-----
printf "Starting new $0 with PID = %4d and PPID = %4d\n" $$ $PPID >
/dev/tty

#-----
# Read a line from STDIN, and parse it
#-----
read line
left=$( echo $line | cut -d: -f1)
pleft=$(echo $line | cut -d: -f2)

#-----
# Write this 'down' the pipeline
#-----
echo "$$: $PPID"

```

\$ startPipeTest

```

Starting ./startPipe with PID = 1888 from shell with PID =
1736
Starting new ./pipeTest with PID = 1896 and PPID = 1888
Starting new ./pipeTest with PID = 1848 and PPID = 1888
Starting new ./pipeTest with PID = 404 and PPID = 1888

```

Note that each of the pipeTest instances invoked from the startPipe script is a child of the startPipe.

11 Control Structures

Control structures determine flow of control in the shell. To understand control structures, it is important to understand a command's exit status.

11.1 Exit Status

When a command is run, its exit status is stored in the *special* variable `$?` . A return value of 0 signifies success whereas a non-zero return value implies failure. It is populated with the value of the return statement, if called from within a function, or the exit statement if called from a script.

For example:

```
$ ls nonExistentFile

nonExistentFile: No such file or directory

$ echo $?

2
```

which indicates failure, since it is non-zero.

Now examine the script *exitTest*

```
$ cat exitTest
```

```
#!/bin/bash
#=====
# File           : exitTest
# Description    :
#
# Revision History:
#
#   28 Jan 2003 - bmarshal
#       Initial Version.
#=====
#-----
# exit value is $1 (the first command line argument) if defined,
# else 0
#-----
exit ${1-0}
```

```
$ exitTest 5; echo $?
```

5

unequivocally shows that the value of `$?` is indeed determined by the `exit` builtin.

11.2 Test Command

The `test` command is used to evaluate an expression and set its exit status to indicate whether an expression is true or false. This makes `test` command useful in conjunction with conditional execution, such as the `if` and `while` statements. For legibility, the `[ ]` builtins were introduced that have exactly the same functionality as the `test` command. The Korn and Bash shells have another test operator: the *double bracket test operator* `[[ ]]` which is slightly different. Let's concentrate on the `test` and `[ ]`, which is equivalent, first.

Tests can be divided into three broad categories: file tests, string tests and numeric tests.

11.2.1 File tests

<code>-e file</code>	true if the file exists
<code>-r file</code>	true if the file is readable
<code>-w file</code>	true if the file is writable
<code>-x file</code>	true if the file is executable
<code>-f file</code>	true if the file is an ordinary file
<code>-d file</code>	true if the file is a directory
<code>-s file</code>	true if the file size is greater than 0
<code>-t file</code>	true if the file is associated with a terminal

11.2.2 String tests

<code>-z str</code>	true if the string is empty (``)
<code>-n str</code>	true if the string is non-empty
<code>str</code>	true if the string is non-empty
<code>str1 = str2</code>	true if str1 and str2 are the same
<code>str1 != str2</code>	true if str1 and str2 are not the same
<code>str1 > str2</code>	true if str1 is lexicographically larger than str2
<code>str1 < str2</code>	true if str1 is lexicographically smaller than str2

11.2.3 Numeric tests

<code>num1 -eq num2</code>	true if num1 and num2 are equal
<code>num1 -ne num2</code>	true if num1 and num2 are not equal
<code>num1 -gt num2</code>	true if num1 is greater than num2
<code>num1 -ge num2</code>	true if num1 is greater than or equal to num2
<code>num1 -lt num2</code>	true if num1 is smaller than num2
<code>num1 -le num2</code>	true if num1 is smaller than or equal to num2

11.2.4 Boolean operators

-a	and operator
-o	or operator
!	negation (not) operator

Some examples:

```
$ test -r /etc/passwd
$ echo $?           # /etc/passwd is readable
```

0

```
$ test -w /etc/passwd
$ echo $?           # /etc/passwd is not writable
```

1

Tests are often used with the AND and OR execution operators, which supply ‘short circuit’ functionality:

```
$ [ -e .profile ] && cat .profile
```

tests whether the file `.profile` exists, and only if so, *cats* it, or

```
$ test -d myDir && cd myDir
```

which only changes directory to `myDir` if it exists as a directory, or

```
$ [ -n $name ] || read -p "Enter your name: " name
```

which prompts the user for a name, but only if the name variable is empty.

It is best to double-quote string variables when used with `test` command to test binary operations. Thus in:

```
$ [ $name1 < $name2 ] && echo "$name1 is lexicographically prior to $name2"
```

it is best to double quote both strings. Otherwise, in the event `$name1` is empty or not set, the test would evaluate as `[ < $name2 ]` and the shell could (depending on the shell) complain with a message of the sorts ‘test: argument expected’.

11.2.5 Caveats

Some guidance on using the `test` or its equivalent `[ ]` operator:

- There must be at least one space after the '[' and before the ']'
- Double quote variables used within tests
- Keep in mind that string and numeric variables use different operators. Originally all shell variables were treated as strings. So strings are first class citizens in *sh* and as such get the nicer operators.
- Within `[[ ]]` tests, one must use the POSIX `||` operator instead of the `-o` operator

11.3 Extended Test Command

The Korn shell first introduced the `[[ ]]` test operator, which Bash adopted. It did this instead of changing the functionality of the original `[ ]` operator, to extend the functionality of the *test* command without sacrificing backward compatibility.

Additional useful tests that `[[ ]]` support include:

```
-a file      true if file exists.
-b file      true if file exists and is a block special file.
-c file      true if file exists and is a character special file.
-d file      true if file exists and is a directory.
-e file      true if file exists.
-f file      true if file exists and is a regular file.
-g file      true if file exists and is set-group-id.
-h file      true if file exists and is a symbolic link.
-k file      true if file exists and its ``sticky'' bit is set.
-p file      true if file exists and is a named pipe (FIFO).
-r file      true if file exists and is readable.
-s file      true if file exists and has a size greater than zero
-t fd        true if file descriptor fd is open and refers to a
             terminal
-u file      true if file exists and its set-user-id bit is set.
-w file      true if file exists and is writable.
-x file      true if file exists and is executable.
-O file      true if file exists and is owned by the effective
             user id.
-G file      true if file exists and is owned by the effective
             group id.
-L file      true if file exists and is a symbolic link.
-S file      true if file exists and is a socket.
-N file      true if file exists and has been modified since it
             was last read.

file1 -nt file2    true if file1 is newer (according to
                   modification date) than file2.
file1 -ot file2    true if file1 is older than file2.
file1 -ef file2    true if file1 and file2 have the same device
                   and inode numbers.
-o optname        true if shell option optname is enabled.
-z string         true if the length of string is zero.
-n string         true if the length of string is non-zero.

string1 == string2 true if the strings are equal.  = may be
                   used in place of ==.
string1 != string2 true if the strings are not equal.
```

<code>string1 < string2</code>	true if <code>string1</code> sorts before <code>string2</code> lexicographically in the current locale.
<code>string1 > string2</code>	true if <code>string1</code> sorts after <code>string2</code> lexicographically in the current locale.
<code>arg1 OP arg2</code>	OP is one of <code>-eq</code> , <code>-ne</code> , <code>-lt</code> , <code>-le</code> , <code>-gt</code> , or <code>-ge</code>
<code>expr1 -a expr2</code>	logical "AND"
<code>expr1 -o expr2</code>	logical "OR"

11.4 If Statement

The syntax of the if statement is:

```
if someCommand
then
    commands
elif someOthercommand
then
    otherCommands
else
    yetOtherCommands
fi
```

The *then* keyword has to be separated from the command. It is typically put on the line below the *if*. It can be put after the command associated with the *if* keyword by terminating the command with a ';' character.

A one-line *if* statement can be written by putting a ';' before the words *then* and *fi*. No semicolon is necessary between the keyword *then* and *commands*.

Note the *fi* reserved word. This is a vestige of the ALGOL programming language.

It is important to note that, unlike most other languages, the if statement actually operates on a *command*, not an expression. Examine the following example:

```
#-----  
# grep looks for a pattern in a file and returns all lines of file  
# that match pattern. If we're not interested in the lines that  
# match, but rather whether or not pattern is in file then throw the  
# output out by redirecting it to /dev/null  
#-----  
if grep pattern file > /dev/null  
then  
    echo "pattern was found in file"  
else  
    echo "pattern was not found in file"  
fi
```

Here the `grep` command is issued. If the command fails – either because the pattern doesn't exist in the file, or because the file is not readable by the user – the *if* fails and the *else* branch is executed.

To compare values or check for file types, use the **test** command:

```
#-----  
# Read input from stdin. The -p option displays a prompt (bash  
# specific). In the korn shell, the equivalent is:  
# read name?"Enter your name: "  
#-----  
read -p "Enter your name: " name  
if test $name = 'Mario'  
then  
    echo "Hello $name"  
else  
    echo "Who art thou?"  
fi
```

or the slightly more legible `[ ]` builtins, a notation introduced in a later version of the Bourne shell:

```
#-----
# The bracket operators were introduced in the Bourne Shell (sh)
# to make the if statement more intuitive. Note, though, that
# there must be a space after the if, after '[' and before # ']'
#-----
read -p "Enter your name: " name
if [ $name = 'Mario' ]
then
    echo "Hello $name"
else
    echo "Who art thou?"
fi
```

The Korn shell introduced, and the Bash shell adopted, the `[[ ]]` keywords to perform string and file checks and `(( ))` to perform numeric tests. These keywords allow a broader spectrum of tests as well as added functionality and should be preferred over *test* and `[]`, for reasons we will see.

The multitude of ways tests can be performed is confusing. White space between the *if* and the bracket and between the bracket and the command, for example, is important and varies with the notation. Furthermore, numeric comparisons may differ from string comparisons, there are two forms of each boolean operator, strings may have to be quoted depending on the notation used for the test. All this is testimony to the fact that the shell's language is not orthogonal .

The following table helps wading through these murky waters.

	Quoting	space required	Logical Operators	
<code>[]</code>	must quote	yes	<code>-a -o</code>	<code>* ? []</code> filename globbing
<code>[[]]</code>	quotes not needed	yes	<code>&& </code>	<code>* ? []</code> pattern expansion
<code>(())</code>	don't quote	no	<code>&& </code>	

```
#-----  
# In this example the '=' is replaced with '=='. Both work, but  
# the later is preferred since it the single '=' is more  
# appropriate for assignment. There must be a space after the if,  
# after '[' and before # ']'  
#-----  
read -p "Enter your name: " name  
if [[ $name == 'Mario' ]]  
then  
    echo "Hello $name"  
else  
    echo "Who art thou?"  
fi
```

Notes on comparing values (more later):

- strings are compared using the operators =, >, >=, <, <=
- numbers are compared using the operators -eq, -ne -gt, -ge, -lt, -le
- the 'if [] ...' syntax was introduced as a more intuitive alternative to 'if test condition ...'
- the newer shells (korn, bash) introduced yet another test operator, the 'double bracket' operator, [[]].

11.5 Case Statement

The syntax of the *case statement* is as follows:

```
case hintVariable in
    pattern [ | pattern ... ] ) commands ;;
    pattern [ | pattern ... ] ) commands ;;
    ...
esac
```

It is a generalized if statement that branches on a *hint*. The case statement compares *hintVariable* to the *patterns* from top to bottom and performs the *commands* associated with the first – and only the first – *pattern* that matches. The *patterns* are written using the shell's pattern matching rules (globs, not regexes). Each action is terminated by the double semicolon ;;

```
#!/usr/bin/bash
# @(#)
#=====
# File           : caseTest
# Description    :
#
# Revision History:
#
#   21 Feb 2003 - marshallb
#       Initial Version.
#=====
#-----
# define three integer variables
#-----
typeset -i lowercase
typeset -i uppercase
typeset -i other

while read line
do
    #-----
    # use the set command to parse $line into tokens stored in $1, $2
    # There are $# tokens in all (the size of $*)
    #-----
    set $line
    for(( i = 0; i < $#; ++i ))
    do
        echo processing $1
        case $1 in
            [a-z]* ) (( ++lowercase )) ;;
            [A-Z]* ) (( ++uppercase )) ;;
            *)      (( ++other )) ;;
```

```
        esac
        #-----
        # remove $1 from $*.  $2 becomes $1, $3 becomes $2 and so on
        #-----
        shift
    done
done

echo "lowercase count = $lowercase"
echo "uppercase count = $uppercase"
echo "other      count = $other"
```

This *filter* reads one line at a time from STDIN, breaks it up into words and counts words that start with a lowercase letter and those starting with an uppercase letter.

```
$ IFS=: cat /etc/passwd | caseTest
```

11.6 Loops

There are occasions when there is a need to execute a number of statements repeatedly until a condition occurs. We present, four looping constructs and two loop control constructs available in the family of Bourne shells.

11.6.1 For Loop

The *for* loop iterates over and operates on a list:

```
for varName in list
do
    cmds
done
```

The keyword *do* must be separated from the list (otherwise it seems as though it were part of the list). Typically it is placed on the next line. It can, however, be placed on the same line as the *for* by putting a ';' character after the list. Likewise, the *done* keyword is typically on a line of its own.

To put the whole loop on one line, place a ';' before the words *do* and *done*. No semicolon is necessary between the keyword *do* and *cmds*.

```
#-----
# The bash command seq (bash 2.05) takes one or two arguments.
# range 10 returns the list 1 2 3 4 5 6 7 8 9 10. range 5 10
# returns the list 5 6 7 8 9 10. The function range is defined
below.
#-----
for value in $(range 5 10)
do
    echo $value
done

# prints 5 6 7 8 9 10 (each number on its own line)
```

Here is another example: The notation `$(ls *.sh)` is expanded to the list of all files in the current directory that have an extension of '.sh'.

```
#-----  
# Echo each file in the current directory that has a .sh  
# extension  
#-----  
  
for file in $(ls *.sh)  
do  
    echo $file  
done  
  
# prints all *.sh files (each on its own line)
```

11.6.2 Arithmetic For Loop

Another *for* loop the *bash* shell offers has the following syntax:

```
for (( expr1; expr2; expr3 ))  
do  
    cmds  
done
```

and is surely familiar to all C/C++ programmers. The first expression, *expr1* is evaluated on entrance of the loop. Processing continues until the second, *expr2*, fails. At the bottom of each loop, the third expression, *expr3*, is evaluated.

As a simple example,

```
$ for (( num = 0; num < 5; ++num ))  
do  
    echo $num  
done
```

prints out the numbers 0 through 4. Note that within the double parentheses the variable names are *not* prefixed with the '\$'.

11.6.3 While Loops

The while loop has the following syntax:

```
while command  
do  
    commands  
done
```

Unusual, compared with other programming languages, is that *while* takes a command and not a boolean expression as an argument. As with the *for* loop, the keywords *do* and *done* are typically on a line by themselves, but a one line *while* loop is possible by prefixing the *do* and *done* with a semicolon.

while iterates whilst the command *command* executes successfully.

```
#-----  
# The while loop takes a command as an argument. Here we read  
# into the variable line from STDIN. Read returns failure (1)  
# when it reads EOF. Since myFile is input redirected, it reads  
# a line from that file.  
#-----  
while read line  
do  
    echo $(( i += 1 )) $line  
done < myFile  
  
# prints the lines of myFile and prepends the line number
```

Another example:

```
#-----  
# Continue looping while the ping command fails. Wait 3 seconds  
# before giving up, then sleep 5 seconds before trying again.  
#-----  
while ! ping -I 3 $host  
do  
    echo "Connecting ..."  
    sleep 5  
done  
  
echo "Connection to $host established"
```

Here the '!' negates the result of the *ping* command.

11.6.4 Until Loops

The until loop has the following syntax:

```
until command  
do  
    commands  
done
```

Unusual, compared with other programming languages, is that *until* takes a command and not a boolean expression as an argument.

until loops until *command* executes successfully.

```
#!/usr/bin/bash
#=====
# File           : untilTest
# Description    :
#
# Revision History:
#
#   10 Feb 2003 -
#       Initial Version.
#=====
#-----
# loop until the user enters a non-empty fileName ( $fileName or
# -n $fileName ) and a file of that name doesn't exist.
#-----
until [[ $fileName && ! -e $fileName ]]
do
    read -p "Enter a file name: " fileName
    if [[ -e $fileName ]]
    then
        read -p "$fileName exists. Overwrite? " response
        #-----
        # does $response start with a 'y' or a 'Y'? Note the
        # wildcard
        #-----
        if [[ $response == [yY]* ]]
        then
            break
        fi
    fi
done
echo "Operating on $fileName"
```

11.6.5 Break statement

The break statement exits the innermost loop in which it is contained. Optionally, break takes an integer as a parameter, which specifies how many loops to jump out of, starting from the innermost loop:

```

#!/usr/bin/bash
# @(#)
#=====
# File           : breakTest
# Description    :
#
# Revision History:
#
#   11 Feb 2003 -
#       Initial Version.
#=====
for i in $(range 5)
do
    for j in $(range 5)
    do
        if (( i == 2 ))
        then
            #-----
            # this will break out of the inner most loops
            #-----
            break
        elif (( i == 4 ))
        then
            #-----
            # this will break out of both loops
            #-----
            break 2
        fi
        printf "\$i = $i and \$j = $j\n"
    done
done
done

```

\$ breakTest

```

$i = 1 and $j = 1
$i = 1 and $j = 2
$i = 1 and $j = 3
$i = 1 and $j = 4
$i = 1 and $j = 5
$i = 3 and $j = 1
$i = 3 and $j = 2
$i = 3 and $j = 3
$i = 3 and $j = 4
$i = 3 and $j = 5

```

11.6.6 Continue statement

The *continue* statement forces processing to proceed to the top of the loop for the next iteration, skipping all statements below the *continue* statement. As with the *break*

statement, the *continue* can take an optional integer as a parameter which specifies for how many loops processing should be skipped, starting from the innermost loop.

```
#!/usr/bin/bash
#=====
# File           : continueTest
# Description    :
#
# Revision History:
#
#   11 Feb 2003 -
#       Initial Version.
#=====
while read -p "Enter a file name: " fileName
do
    if [[ -z $fileName || -e $fileName ]]
    then
        continue
    fi
    break
done

echo "Processing file $fileName"
```

In this code fragment, the user is prompted for a file name. If the entered name is the empty string (-z check) or a file of that name already exists (-e check), the user is prompted to enter another file name. Upon exit of the loop, the user is guaranteed to have a non-empty file name for which there is no file.

11.7 Select Statement

The *select* statement has the following syntax:

```
select name in wordList
do
    list
done
```

The list of words, *wordList*, following the keyword *in* is expanded, generating a list of items. The set of expanded words is printed on the standard error, each preceded by a number. If the *in* word is omitted, the positional parameters are printed. The PS3 prompt is then displayed and a line read from the standard input. If the line consists of a number corresponding to one of the displayed words, then the value of *name* is set to that word. If the line is empty, the words and prompt are displayed again. If EOF is read, the command completes. Any other value read causes *name* to be set to null. The line read is saved in the variable *REPLY*. The list is executed after each selection until a *break* command is executed. The exit status of *select* is the exit status of the last command executed in list, or zero if no commands were executed.

Using the select statement, it is easy to write a simple file manager:

```
#!/usr/bin/bash
#####
# Script:  filemanager
# Author:  marshallb
# Purpose: Use the Bash shell select statement to display a list of files
#          and directories in the current directory, or user-specified
#          directory.
#####
#=====#
# Function: getHelp
#=====#
function getHelp
{
    cat <<-EOM
    -----
    Usage: $0 [ -t ] [ -a <param> ] [ -h ]

    where -t trace    ---> enable execution trace
          -a action   ---> what command to perform on file
          -h help     ---> this help screen
    -----
    EOM
    exit 1;
}

#=====#
# Function: selectFiles
#=====#
function selectFiles
{
    #-----#
    # The builtin typeset ensures that the variables dir and action are
    # local to the body of the function.
    #-----#
    typeset dir=$1
    typeset action=$2

    #-----#
    # Set up menu for the user to choose from
    #-----#
    select choice in 'exit' '..' $(ls); do
        case $choice
        in
            exit)
                exit ;;

            * )
                if [[ -f $choice ]] ; then
                    #-----#
                    # eval dynamically executes the string passed to it
                    #-----#
                    eval "$action $choice"
                elif [[ -d $choice ]] ; then
                    dir=$choice
                fi
            fi
        esac
    done
}
```

```

        [[ $dir != '.' ]] && cd $dir
    fi ;;
esac

    selectFiles $dir "$action"
done
}

#=====
# Main execution
#=====
while getopts :ha:t option; do
    case "$option"
    in
        h)    getHelp          # print out help paragraph
              exit 1;;
        a)    action=$OPTARG;; # what command to perform on file
        t)    typeset -ft selectFiles;;
        \?)   print "Usage: [-h] help [-a] action on file [-t] enable trace"
              exit 1;;
    esac
done

#-----
# shift away the command line switches ( -h -a <arg> -t )
#-----
shift $(( OPTIND - 1 ))

#-----
# startDir defaults to '.' if not specified on the command line
#-----
startDir=${1:-.}

#-----
# action defaults to less if not specified on the command line via -a
#-----
: ${action:=less}

#-----
# quote $action since it may consist of more than one word, ala 'ls -l'
#-----
selectFiles $startDir "$action"

```

12 Command Line Arguments

When a script is invoked:

```
$ myScript cmdLineArgOne cmdLineArgTwo cmdLineArgThree
```

the command line arguments are made available to the script in the variable \$1, \$2, ... \$9, and, for the newer shells (ksh, bash) in \${10}, \${11} and so on, for command line arguments 10 and above.

The number of command line arguments passed to the script is captured in the variable \$#. In addition, two special variables, \$* and \$@, have values that consist of *all* command line arguments. Although \$* and \$@ are very similar, they differ in how they deal with quoted command line arguments, as the following script shows:

```
#!/bin/bash
#=====
# File           : args
# Description    :
#
# Revision History:
#
#   11 Feb 2001 - Brian P Marshall
#       Initial Version.
#=====
typeset -i i=0

echo "There are $# command line arguments"
echo 'Here are the different interpretations of the $*, "$*", $@, "$@"
variables'

echo '----- $* -----'
i=0
for a in $* ; do
    echo "$i $a"
    i=$((i+1))
done

echo '----- "$*" -----'
i=0
for a in "$*" ; do
    echo "$i $a"
    i=$((i+1))
done

echo '----- $@ -----'
i=0
for a in $@ ; do
    echo "$i $a"
    i=$((i+1))
done
```

```

echo '----- "$@" -----'
i=0
for a in "$@" ; do
    echo "$i $a"
    i=$((i+1))
done

```

Now examine the output of the script:

\$ args one 'two three' "four five" six

```

There are 4 command line arguments
Here are the different interpretations of the $*,
"$*", $@, "$@" variables
----- $* -----
0 one
1 two
2 three
3 four
4 five
5 six
----- "$*" -----
0 one two three four five six
----- $@ -----
0 one
1 two
2 three
3 four
4 five
5 six
----- "$@" -----
0 one
1 two three
2 four five
3 six

```

Firstly, notice that quotes – single or double – delimit command line arguments. When the shell executes the above script, the shell passes four command line parameters to the script *and then strips the quotes*. Within the script, the expressions `$*` and `$@` lose track of quotes, whereas the expression `"$@"` doesn't.

13 Regular Expressions

Regular expressions form a meta-language that can be used to describe patterns. Many Unix utilities – including `grep`, `sed`, `awk`, `vi` -- use *regular expressions*. The following characters have special meaning in regular expressions:

.	matches any character except '\n'
*	0 or more of the previous character
\	escapes the special meaning of an RE character
^	matches the beginning of a string
\$	matches the end of the string
\<	matches the beginning of a word
\>	matches the end of a word
[]	matches any one character from the specified set
[-]	matches any one character in the specified range
[^]	matches any one character NOT from the specified set

The following examples help clarify the above notation:

```
$ grep '...' fileName
```

matches all lines of *fileName* that have at least three characters.

```
$ grep '^aa*.*b$' fileName
```

matches all lines of *fileName* that start with at least one 'a', followed by any number of characters and end in 'b'.

```
$ grep '[A-Za-z][A-Za-z]*[0-9A-Za-z]' fileName
```

matches all lines of *fileName* that have one or more alphas followed by an alphanum.

Examine the following file:

```
$ cat data
```

```
a
aa
aaa
ababa
abaabbbaabbbaa
```

Using the *hlp* function described below, which highlights matched patterns passed in as a parameter, we observe:

```
$ cat data | hlp '...'
```

```
a
aa
aaa
ababab
abaabbaaabbbbaa
```

Multiples of three characters are highlighted (bolded in this document)

```
$ cat data | hlp 'b.*b'
```

```
a
aa
aaa
ababab
abaabbaaabbbbaa
```

Here we are matching the strings starting and ending with 'b', with any characters in between. Note that for the last line, the pattern did not match 'baab' - the smallest string satisfying the regex 'b.*b', but rather 'baabbbaabbb', which is the largest string satisfying the regex. Because of this feature, regular expressions are called *greedy*.

```
$ cat data | hlp 'bbb*'
```

```
a
aa
aaa
ababa
abaabbbaaabbbbaa
```

Here we are matching strings that consist of at least two 'b's since the regex 'bbb*' means 'b' followed by 'b' followed by 0 or more 'b's.

It is often useful to capture a *pattern match* in order to be able to use it in a substitution. A pattern match is captured by placing parentheses around the pattern. Since these have a special meaning in the shell, they often have to be escaped.

14 Processes

The two main concepts in *Unix* are *processes* and *files*.

A process is a program in execution. It is created when an executable file is started from the command line and consists of a stack, a heap, an environment and a text segment, which contains the executable code.

A process has a *process id*, abbreviated as *PID* that uniquely identifies it. It is identified by the shell variable `$` or `PID`, which have one and the same value:

```
echo "\$ = $$ and \$PID = $PID"
```

```
$ = 2516 and $PID = 2516
```

```
$ ps -ef
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	0	0	0	Jan 26	?	0:01	sched
root	1	0	0	Jan 26	?	0:39	/etc/init -
root	2	0	0	Jan 26	?	0:00	pageout
root	3	0	1	Jan 26	?	1198:12	fsflush
root	75	1	0	Jan 26	?	0:07	devfsadmd
hnagaraj	25107	25066	0			0:02	<defunct>
root	211	1	0	Jan 26	?	0:00	/usr/lib/sunscreen/lib/ss_logd
root	56	1	0	Jan 26	?	0:00	/usr/lib/sysevent/syseventd
root	773	1	0	Jan 26	?	0:00	/usr/lib/saf/sac -t 300

lists in full (-f) every (-e) running process. Note that the *PPID* – the processes’ parent process id – is 0 for the first four processes. These are the so called *spontaneous* processes that are started when the *Unix* box is started.

All of the non-spontaneous processes are created by the *fork* system call and have the init process, which always has a *PID* of 1, as an ancestor.

For every running process, there is a directory in the `/proc` filesystem for it. It contains a number of files and directories that describe the program in execution. One such directory is the *fd* directory which contains the list of open file descriptors the file has opened:

```
$ ls -l /proc/$$/fd
```

c-----	1	marshall	tty	24, 12 Feb 12 10:03	0
c-----	1	marshall	tty	24, 12 Feb 12 10:03	1
c-----	1	marshall	tty	24, 12 Feb 12 10:03	2
c-----	1	marshall	tty	24, 12 Feb 12 10:03	255
D-----	1	root	root	0 Jan 26 13:27	3>

When a process is initiated, three files are opened by default. These are `STDIN`, `STDOUT` and `STDERR` with file descriptors 0, 1 and 2 respectively. These visible in the above listing.

14.1 Files

A simple view of a *file* as a container of data is limiting. In *Unix*, it is more appropriate to say that a *file* is anything that can be read from or written to. That makes your screen, the keyboard, the printer or fax a file. More appropriately for the latter category of files, there are device drivers that are invoked with the code to read from or write to the device in question. This device driver is associated with a device file typically located in the `/dev` directory.

14.2 Input Redirection

The `read` command reads from *STDIN*:

```
$ read userName passwd uid remainder
```

reads from *STDIN* into the four variables specified after the `read`. The environmental variable `IFS` – the internal field separator – contains the token delimiters. In the above example, `userName` will contain the first word, `passwd` the second, `uid` the third word and `remainder` will contain the remaining words.

To read from another file, we use the *input redirection mechanism*

```
$ OLD_IFS="$IFS"
$ IFS=:
$ read userName passwd uid remainder < /etc/passwd
$ IFS="$OLD_IFS"
```

gets its input from the `/etc/passwd` file. The `<` is the *input redirector*. Before the `read` command is executed, input redirection occurs: a *file handle* is created for reading and associated with `/etc/passwd` with same descriptor as *STDIN*. Thus reading from *STDIN* actually reads from the `passwd` file.

14.3 Here Documents

Here documents have the following syntax:

```
<<[-] EndOfDataToken
line1
...
linen
EndOfDataToken
```

This type of redirection instructs the shell to read input from the current source, starting the line after the token `<<` until a line containing only *EndOfDataToken* (with no

trailing blanks) is seen. All of the lines read up to that point are then used as the standard input for a command.

The `'-'`, optionally specified before the *EndOfDataToken* directs bash to strip leading tabs from the *Here Document*.

Here documents allow a programmer to embed data directly in a shell script. Note that *variable expansion and command substitution do occur* within such a document, unless *end of data marker* is single quoted.

The following script uses a *Here Document* to create a self extracting archive that bundles all files passed in as command line arguments.

```
#!/usr/bin/bash
#=====
# File      : bundle
# Description : This script creates a self extracting archive
#              that bundles all files passed in as command line
#              arguments. Executing this archive extracts the
#              bundled files.
# Usage     : bundle archiveName files
#
# Revision History:
#   11 Oct 2001  marshallb
#       Initial Version.
#=====
#-----
# The archive is first command line argument. Grab it or exit with a
# usage statement.
#-----
archFile=${1:? "Usage: $0 archiveFile file(s)"}

#-----
# shift the archiveFile away
#-----
shift

#-----
# This is the Here Doc. The '-' before the marker EOF tells bash to strip
# leading tabs from the doc. Note that variable expansion and command
# substitution occur within such a document.
#-----
cat <<-EOF > $archFile
#! $(which bash)
#=====
# File      : $archFile
# Description : This is a self extracting archive file
#
# Revision History:
#   $(date +"%d %h %Y") $(grep ^$LOGNAME: /etc/passwd | awk -F: '{print
# $5}')
#       Initial Version.
#=====
EOF
```

```
for f
do
    delim=$(basename $f)EOF
    #-----
    # The archive consists of a number of Here Documents which autoextract
    # into one file per bundled file.
    #-----
    echo "cat <<-$delim > $f" >> $archFile
    cat $f >> $archFile
    echo $delim >> $archFile
done

chmod +x $archFile
```

14.4 Output Redirection

The default output file is STDOUT. Thus:

```
$ ls
```

produces a listing of all files in the current directory and writes it to STDOUT. To write the output to file of your choosing instead, the *output redirection* mechanism is used:

```
$ ls > listing
```

This creates the file *listing*, overwriting its previous contents⁷, *but* using the file descriptor of STDOUT. Thus writing to STDOUT – the file with descriptor 1 – has the effect of writing to *listing*.

14.5 Output Appending

To append the output of a command to a file, which needn't exist, use the '>>' mechanism:

```
$ date >> myFile
$ pwd >> myFile
```

appends the output of *pwd* to *myFile*. Note that that file will be created on the line above if it doesn't already exist.

14.6 Opening a file with a given file descriptor

⁷ unless *noclobber* is set (set `-o noclobber` from the command line). In the case *noclobber* is set, a redirection can still be forced using the notation:

```
$ls >| listing
```

A faction of shell programmers consider it poor style to deal with the low-level concept of a file descriptor. Reading and writing files should occur, they maintain, through re-direction alone.

There are occasions, however, in which it is useful to open a file and assign it a specific file descriptor. This can be accomplished using *exec*:

```
$ exec 4 < inputFile
$ exec 5 > outputFile
```

inputFile is opened for reading and is assigned file descriptor 4; *outputFile* is opened for writing and assigned file descriptor 5.

We can verify this by remembering that the *fd* subdirectory of *\$\$*, the PID of the current process, contains an entry for each of the process' open files:

```
$ls -l /proc/$$/fd

D----- 1 root      root          0 Jan 26 13:27 3>
-r--r--r-- 1 marshall staff        13 Feb 11 09:53 4
--w----- 1 marshall staff          0 Feb 12 11:59 5
c----- 1 marshall tty          24, 66 Feb 12 11:59 6
c----- 1 marshall tty          24, 66 Feb 12 11:59 255
c----- 1 marshall tty          24, 66 Feb 12 11:59 2
c----- 1 marshall tty          24, 66 Feb 12 11:59 1
c----- 1 marshall tty          24, 66 Feb 12 11:59 0
```

We see that 4 is opened for reading, whereas 5 is opened for writing.

14.7 Closing a file with a given file descriptor

A file with descriptor 4 opened for reading as above is closed using the *exec* command :

```
$ exec 4<&-
```

Likewise, to close the file with descriptor 5 opened for writing, issue::

```
$ exec 5>&-
```

A quick sanity check shows they are indeed closed:

```
$ls -l /proc/$$/fd

D----- 1 root      root          0 Jan 26 13:27 3>
c----- 1 marshall tty          24, 66 Feb 12 11:59 6
c----- 1 marshall tty          24, 66 Feb 12 11:59 255
c----- 1 marshall tty          24, 66 Feb 12 11:59 2
```

```
c----- 1 marshall tty      24, 66 Feb 12 11:59 1
c----- 1 marshall tty      24, 66 Feb 12 11:59 0
```

14.8 File Merging

The output or input of two files can be merged. The typical idiom is:

```
$ if myScript > /dev/null 2>&1
  then
    do something ...
  fi
```

Here the output of *myScript* isn't needed or wanted and hence it is re-directed to the *bit-bucket*. At the same time, STDERR is merged with STDOUT, as indicated by `2>&1`, with the result that errors normally written to STDERR are discarded instead.

Given two files with descriptors of *m* and *n* respectively, the output of the first file is merged with the output of the second using the syntax:

```
m>&n
```

14.9 Reading from a file

The following code segment shows a common way of reading from a file:

```
$ while read name
  do
    read -p "$name, enter your age: " age < /dev/tty
    printf "%10s is %2d years old\n" $name $age >> namesAndAge
  done < namesFile
```

Note that *read*, that normally reads a line from STDIN is now reading a line from *namesFile* because of the input redirection. This forces the *read* within the body of the *while* loop to explicitly read from */dev/tty*, the file associated with the terminal, lest it read from *namesFile* too.

If a script has to simultaneously read from several files, using file descriptors is useful:

```

#!/usr/bin/bash
#=====
# File           : readTest
# Description    : This script shows how to read from three files
#                  simulataneously (STDIN being the third) without
#                  having to redirect.
#
# Revision History:
#
#   13 Feb 2003 - marshallb
#       Initial Version.
#=====
#-----
# open the two passed in files for reading
#-----
exec 5<${1:?Usage: $0 <file1> <file2>}
exec 6<${2:?Usage: $0 <file1> <file2>}

#-----
# loop forever. The builtin : is a NOOP that always returns true
#-----
while :
do
    #-----
    # note that we can read from the terminal without having to
    # redirect /dev/tty
    #-----
    read -p "Do you want to continue (y/n)? " -n 1 response
    if [[ $response != 'y' ]]
    then
        break
    fi
    #-----
    # read a line from each of the files ...
    #-----
    read line1 0<&5
    read line2 0<&6
    #-----
    # ... until each read in line is empty
    #-----
    [[ -z $line1 && -z $line2 ]] && break
    echo "<$line1> ===== <$line2>"
done

#-----
# close the files
#-----
exec 5<&-
exec 6<&-

```

The advantage here that we can read from more than one file simultaneously without redirection.

14.10 Writing to a file

Output redirection is considered the preferred way of writing to a file:

```
$ date > data
$ pwd >> data
```

writes to lines to the file *data*.

On occasions, however, it can be useful to write to the file associated with a given file descriptor. Unlike the Korn Shell, which has direct facilities for reading and writing from and to files:

```
$ read -u 5 name    # ksh only: read into name from file descriptor 5
$ print -u 6 $name  # ksh only: write $name to file descriptor 6
```

bash has no such capability. However, the common idiom:

```
$ exec 5 > myFile    # open myFile for writing and associate it with descriptor 5
$ exec 1 >& 5         # merge STDOUT with myFile
$ echo "Hello there" # output is written to myFile
$ echo Bye>/dev/tty # this is written to the terminal
$ exec 5>&-          # close the file with descriptor 5 (myFile)
```

does redirect any output intended for STDOUT to *myFile* since the second line of the above sequence merges STDOUT with *myFile*. There two are drawbacks however. Firstly, any output destined for STDOUT will now be written to the file instead, unless one *explicitly* writes to the terminal, */dev/tty*, as the example shows. Secondly, code needs to be added to reinstate STDOUT once *myFile* is closed. So the complete functional example is:

```
$ exec 5 > myFile 1>&5    # combine steps 1 and 2 above
$ echo "Hello there"    # output is written to myFile
$ echo Bye > /dev/tty    # this is written to the terminal
$ exec 5 > &-           # close the file with descriptor 5 (myFile)
$ exec 1 > /dev/tty      # restore STDOUT
```

15 Functions

A *shell function* defines a set of commands that are executed when the function name is invoked.

There are two ways to define a function:

```
function functionName
{
    code
}
```

or

```
functionName()
{
    code
}
```

The closing brace must be on a line itself or separated from the function body by a semicolon. Thus one-line function definitions are possible:

```
$ function bold { tput bold ; }
```

Like scripts, functions can be invoked using command substitution. Unlike scripts, functions are retained in memory so they don't need to be loaded each time they are called.

The following command lists all loaded functions:

```
$ typeset -f
```

Parameters can be passed to functions:

```
$ myFunction paramOne paramTwo paramThree
```

and return values from functions captured using command substitution:

```
$ result=$(myFunction paramOne paramTwo paramThree)
```

The following script named *recurse* contains a function of the same name. It can be invoked by passing in a command. The passed in command operates on each subdirectory:

```
$ cat recurse
```

```

#!/usr/bin/bash

#=====
# Function      : recurse
# Description: Walk a directory depth first and execute a
#              command in each directory
#=====
function recurse
{
    #-----
    # local insures that the following assignment doesn't
    # change the value of some global
    #-----
    local cmd=$1
    #-----
    # Perform a long listing and choose those files that
    # start with the letter 'd'. These are the directories.
    # Then use awk to print the last field ($NF). Command
    # substitution guarantees that the ls command itself
    # is replace with the list of files it returns
    #-----
    for dir in $(ls -l | grep ^d | awk '{print $NF}')
    do
        cd $dir
        #-----
        # Which recurse gets called? The function, not
        # the script. See Parsing Order elsewhere in this
        # document.
        #-----
        recurse "$cmd"
        #-----
        # cmd is a variable whose value is a command string.
        # The eval builtin takes a string and executes
        # it. As an example: cmd=ls; eval $cmd
        # lists the files in the current directory.
        #-----
        eval $cmd
        cd ..
    done
}

recurse "$1"

```

`$ recurse 'pwd ; date'`

Prints the current working directory followed by the date for each subdirectory.

As the script shows, functions have precedence over scripts of the same name.

In order to keep a function in memory, place its code in a file – typically of the same name as the function itself – and source that file. If at any point the code of the function is changed, the function has to be unset before the file containing the code is sourced:

```
$ unset -f functionName
$ . functionName
```

In the Korn shell, the environmental variable FPATH is a colon separated list of directories the shell examines in searching for a function. It looks for a file of the same name as the function and sources it. Bash, unfortunately doesn't offer this facility.

Therefore, if there are functions that you would like to have available at every login, put the source of those functions in a directory and source every file in that directory in your login script (.profile or .bash_profile).

Some useful functions

Here are some useful functions with examples:

15.1 ***bold***

The function *bold* turns **bold** on.

```
#=====
# Function: bold
# This function turns bold font on
#=====
function bold
{
    tput bold
}

declare -fx bold
```

15.2 ***boldOff***

The function *boldOff* turns **bold** off.

```
#=====
# Function: boldOff
# This function turns bold font off
#=====
function boldOff
{
    tput rmso
}

declare -fx boldOff
```

```
$ echo "Hello there $(bold)Sarah$(boldOff). How are you?"
```

```
Hello there Sarah.  How are you?
```

15.3 *hlt*

The function *hlt* turns **highlighting** on.

```
#####  
# Function: hlt  
# This function turns highlighting on  
#####  
function hlt  
{  
    tput smso  
}  
  
declare -fx hlt
```

15.4 *hltOff*

The function *hltOff* turns **highlighting** off.

```
#####  
# Function: hltOff  
# This function turns highlighting off  
#####  
function hlt  
{  
    tput rmso  
}  
  
declare -fx hltOff
```

15.5 *hlp*

The *hlp* (highlight pattern) function is useful when searching for patterns in a file. It takes a quoted regular expression and highlights every occurrence of that pattern in the named files or from STDIN if no files are passed as parameters. It is also helpful in understanding regular expressions.

```
#####  
# Function      : hlp (highlight pattern)  
# Usage        : hlp <regEx> [files]  
# Description: hlp is a filter.  It highlights every occurrence of  
#               the passed in pattern within the named files or  
#               from STDIN if no files are passed as parameters  
#####  
function hlp  
{  
    local pattern=$1  
    shift          # shift away the pattern
```

```

if [[ -n $1 ]]      # are there any filenames?
then
    sed "s|\($pattern\)|$(hlt)\l$(hltOff)|g" $*
else
    sed "s|\($pattern\)|$(hlt)\l$(hltOff)|g"
fi
}

```

It reads from STDIN and highlights the pattern passed in as a command line argument. This pattern can be any regular expression *sed* understands. The following example shows how to use this function to advantage in searching files for one pattern and then highlighting a second pattern in the filtered stream:

```
$ grep while * | hlp read
```

```

files:      for tok in $(while read line; do echo
$line; done);
files:      while read line; do
files:          while read line; do
hlp:  while read line
readm:      for tok in $(while read line; do echo
$line; done);
sws:      while read line; do
tev:          while read line; do

```

Highlighting can make it much easier to find a pattern in reams of data.

15.6lower

The function *lower* takes a string as an argument and returns it in lowercase.

```

#=====
# Function: lower
#=====
lower ()
{
    echo $1 | tr A-Z a-z
}
declare -fx lower

```

```
$ id=$(read -p "Enter your id: " id; lower $id)
```

In this example, a user is prompted to enter her id. Command substitution guarantees that the invoked command is replaced with its output, which is *\$id* in lower case. Note that the prompt 'Enter your id' is not returned: indeed this prompt is written to STDERR and hence isn't captured by command substitution.

15.7 upper

The function *lower* takes a string as an argument and returns it in uppercase.

```
#####
# Function: upper
#####
upper ()
{
    echo $1 | tr a-z A-Z
}
declare -fx upper
```

```
$ id=$(upper $(read -p "Enter your id: " id; echo $id))
```

This example is a variation on the theme of the above example with the difference that here we have nested *command substitutions*.

15.8 initCap

The function *initCap* takes a string as an argument and returns it with the first character in uppercase.

```
#####
# Function: initCap
#####
function initCap
{
    echo "$ (expr $1 : '\(.\)') | tr [a-z] [A-Z]) $(expr $1 :
'.\(.*\)')"
```

```
declare -fx initCap
```

15.9 getKey

The *getKey* function reads exactly one keystroke and returns the entered character. It does this by changing the terminal settings to *raw* mode, which is unbuffered, and reading exactly one (count=1) block of size 1 (bs=1) from /dev/tty (STDIN) and returning it. Echo'ing is turned off before the character is read and turned back on before exiting the function.

```
#####
# Function: getKey
#####
getKey ()
{
    local ch;
    local option;
    option=$1;
```

```

stty raw;
stty -echo;
ch=$(dd if=/dev/tty bs=1 count=1 2>/dev/null);
stty -raw;
stty echo;
case $option in
    -l)
        ch=$(lower $ch)
        ;;
    -u)
        ch=$(upper $ch)
        ;;
esac;
echo $ch
}

declare -fx getKey

```

15.10sws

The function *sws* (*squeeze white space*) is a filter that removes duplicate white space from its input stream. It is meant to be used in a pipeline.

```

#=====
# Function: sws
#=====
function sws
{
    while read line
    do
        echo $line
    done
}
declare -fx sws

```

See section 17.1 for an example of using this function.

15.11lf

The function *lf* (*last file*) takes an integer as an argument and lists that many of the newest files. If no command line arguments are supplied, it displays the *last file*.

```

#=====
# Function: lf
#=====
function lf
{
    ls -ltr | tail -${1:-1}
}
declare -fx lf

```

15.12cf

The function *cf* (*cat file*) takes an integer as an argument and *cats* that many of the newest ordinary files. If no command line arguments are supplied, it *cat s* the *last file*.

```
#####
# Function: cf
#####
function cf
{
    cat $(ls -ltr | grep -v ^d | awk '{print $NF}' | tail -${1:-1})
}
declare -fx cf
```

15.13vf

The function *vf* (*vi file*) takes an integer as an argument and *vis* that many of the newest ordinary files. If no command line arguments are supplied, it *vi s* the *last file*.

```
#####
# Function: vf
#####
function vf
{
    vi $(ls -ltr | grep -v ^d | awk '{print $NF}' | tail -${1:-1})
}
declare -fx vf
```

15.14range

The function *range* takes 0, 1 or 2 integer arguments and returns a range of integers. If no arguments are supplied, it returns the range of integers 1 .. 10. If only one argument is supplied, then it returns the range 1 .. arg. Otherwise it returns the range of integers between the two arguments, inclusive.

Some versions of *bash* have the function *seq* that performs a similar task.

```
#####
# Function: range
#####
range ()
{
    typeset lower
    typeset upper
    typeset result

    if (( $# < 2 ))
    then
        lower=1
        #-----
        # There are less than 2 command line arguments. So set
        # lower to 1 and upper to the first command line arg,
```

```
# if it exists, or 10 if it doesn't
#-----
upper=${1:-10}
else
    lower=$1
    upper=$2
fi

if (( lower <= upper ))
then
    increment=1
else
    increment=-1
fi

while (( lower != upper ))
do
    result="$result $lower"
    (( lower += increment ))
done
echo "$result $upper"
}

declare -fx range
```

As an example,

```
$ for i in $(range 5)
do
    ls -ltr
    sleep 2
done
```

performs a long listing every two seconds for a total of five times.

16 Parsing

Rudimentary parsing is possible in the shell. Let us touch on some common techniques.

16.1 echo

A well known, but not often document idiom, is that the *echo* command strips white space from strings. Here white space consist of spaces, tabs, newlines.

```
$ x='AT&T      Labs'  # 10 spaces
$ echo $x
```

```
AT&T Labs
```

The following function, *sws*, Squeeze White Space, is a useful function that makes use of this idiom:

```
#=====
# Function: sws
#=====
function sws
{
    while read line
    do
        echo $line
    done
}
```

sws is a *filter*. Thus it expects input from STDIN and passes output on to STDOUT. The effect of this function is to remove duplicate whitespace. It does this by reading one line at a time from STDIN and applying *echo* – which removes duplicate white space and prints the squeezed string to STDOUT.

Let's use this to calculate the sum of the sizes of all files in the current directory:

```
$ ls -l
```

```
total 42
-rwxrwxrwx  1 marshall unknown    364 Jan 29 16:30
Tab.cpp*
-rwxrwxrwx  1 marshall unknown    311 Jan 29 16:30
abc.cpp*
-rwxrwxrwx  1 marshall unknown    403 Jan 29 16:30
bob.cpp*
```

```

-rwxrwxrwx      1 marshall unknown      480 Jan 29 16:30
bobT.cpp*
-rwxrwxrwx      1 marshall unknown      339 Jan 29 16:30
inTest.cpp*
-rwxrwxrwx      1 marshall unknown      339 Jan 29 16:30
parseT.cpp*
-rwxrwxrwx      1 marshall unknown     7308 Jan 29 16:30
reporter.cpp*
-rwxrwxrwx      1 marshall unknown      421 Jan 29 16:30
sizeTest.cpp*
-rwxrwxrwx      1 marshall unknown      600 Jan 29 16:30
tReporter.cpp*
-rwxrwxrwx      1 marshall unknown      856 Jan 29 16:30
testUS.cpp*
-rwxrwxrwx      1 marshall unknown    10060 Jan 29 16:30
testcasesel.cpp*
-rwxrwxrwx      1 marshall unknown    13376 Jan 29 16:30
tstring.cpp*
-rwxrwxrwx      1 marshall unknown      493 Jan 29 16:30
tt.cpp*

```

so that

```
$ ls | sws
```

```

total 42
-rwxrwxrwx 1 marshall unknown 364 Jan 29 16:30 Tab.cpp
-rwxrwxrwx 1 marshall unknown 311 Jan 29 16:30 abc.cpp
-rwxrwxrwx 1 marshall unknown 403 Jan 29 16:30 bob.cpp
-rwxrwxrwx 1 marshall unknown 480 Jan 29 16:30 bobT.cpp
-rwxrwxrwx 1 marshall unknown 339 Jan 29 16:30 inTest.cpp
-rwxrwxrwx 1 marshall unknown 339 Jan 29 16:30 parseT.cpp
-rwxrwxrwx 1 marshall unknown 7308 Jan 29 16:30
reporter.cpp
-rwxrwxrwx 1 marshall unknown 421 Jan 29 16:30 sizeTest.cpp
-rwxrwxrwx 1 marshall unknown 600 Jan 29 16:30
tReporter.cpp
-rwxrwxrwx 1 marshall unknown 856 Jan 29 16:30 testUS.cpp
-rwxrwxrwx 1 marshall unknown 10060 Jan 29 16:30
testcasesel.cpp
-rwxrwxrwx 1 marshall unknown 13376 Jan 29 16:30
tstring.cpp
-rwxrwxrwx 1 marshall unknown 493 Jan 29 16:30 tt.cpp

```

removes redundant whitespace. This allows us to pass this information on to the *cut* command (see below) to parse out the fifth space-delimited token.

```
$ ps | sws | cut -d' ' -f5
```

```

364
311
403
480
339
339

```

```

7308
421
600
856
10060
13376
493

```

Now the *echo idiom* can be used again

```
$ echo $( ps | sws | cut -d' ' -f5)
```

```
364 311 403 480 339 339 7308 421 600 856 10060 13376 493
```

to remove whitespace (including newlines) producing one single line of data. The *sed* command (see below) can subsequently be invoked to substitute each (hence the ‘g’ character in the sed script) space (‘ ’) with a ‘+’ character:

```
$ echo $( ps | sws | cut -d' ' -f5) | sed 's/ /+/g'
```

```
364+311+403+480+339+339+7308+421+600+856+10060+13376+493
```

The result is an arithmetic string that can be fed to the Unix *bc* (basic calculator) command to give us the result:

```
$ echo $( ps | sws | cut -d' ' -f5) | sed 's/ /+/g' | bc
```

```
35350
```

In true Unix style, the last sequence of commands can now be used in a *command substitution* to give the user a somewhat meaningful message:

```
$ echo "The files in $PWD take up $(echo $( ls -l | sws | cut -d' ' -f5) | sed 's/ /+/g' | bc) bytes"
```

```
The files in /tmp/t take up 35350 bytes
```

Notice the nested commands. In the days of the Bourne shell, this would have been harder since nested back quotes requires escaping.

16.2 set

The `set` command can be used to parse a string. It takes a string – either a literal or the results of *command substitution* – and breaks the string into words stored in the *positional parameters* `$1`, `$2` The sum total of all parsed words is stored in `$*`, and the total number of parsed words in `$#`.

Tokens are delimited by the characters in the environmental variable IFS (Internal Field Separator).

Thus the code segment prints, for each user, the shell he uses.

```
#!/usr/bin/bash
#=====
# File           : showShell
# Description    :
#
# Revision History:
#
#   30 Jan 2003 - marshallb
#       Initial Version.
#=====
#-----
# Normally it is a good idea to store away the value of IFS and
# restore it upon completion:  $OLD_IFS="$IFS"; ...  $IFS="$OLD_IFS
# Note that since IFS may contain a space, it is quoted.  Within
# a script as this it isn't necessary since it is only set in
# a subprocess of the shell.
#-----
OLD_IFS="$IFS"
IFS=:

while read line
do
    #-----
    # After the set, the loginName is stored in $1, the shell
    # in $7
    #-----
    set $line
    echo "$1 uses shell $7"
done < /etc/passwd

IFS="$OLD_IFS"
```

16.3tr

The `tr` filter translates one set of characters to another using the syntax

```
tr charSet1 charSet2
```

Character sets can be specified as strings of characters or ranges.

```
$ cat myCrypt
```

```
#!/usr/bin/bash
# @(#) A simple encryption scheme
#=====
# File           : myCrypt
# Description    :
#
# Revision History:
#
#   30 Jan 2003 - marshallb
#   Initial Version.
#=====
#-----
# Bail with a Usage statement if the script is invoked without
# a command line arguments. Otherwise pass those arguments to
# the tr filter, which swaps an 'a' and a 'n', 'b' and 'o',
# 'c' and 'p' and so on.
#-----
echo "${*?"Usage: $0 <string>"}" | tr '[a-m][n-z]' '[n-z][a-m]'
```

Thus:

```
$ myCrypt 'secret message'
```

produces

```
frperg zrffntr
```

and

```
$ myCrypt $(myCrypt 'secret message')
```

gives

```
secret message
```

showing that `myCrypt` is *idempotent*.

Another useful idiom uses *tr* to transform spaces to newline characters, which can be used to count the number of words in a file:

```
$ cat wordFreq
```

```
#!/usr/bin/bash
# @(#) Count the number of occurrences of each word
#=====
#
# File           : wordFreq
# Description    :
#
# Revision History:
#
#   30 Jan 2003 - marshallb
#       Initial Version.
#=====
#
#-----
# Translate a space to a '\n', translate a tab to a '\n'. The
# result is sorted and passed on to uniq, which removes dups.
# The -c option writes a frequency count.
#-----
tr -s ' \t' '\n\n' | sort | uniq -c
```

wordFreq is a *filter* which operates only on STDIN. Hence it must be used in a pipeline:

```
$ cat myFile | wordFreq
```

tr has three command line options worth mentioning:

- -d delete characters in the given character set
- -c operate on the *complement* of the given character set
- -s squeeze out (remove) duplicates in the character set

Thus

```
$ cat file | tr -d 'aeiou'
```

removes all vowels from *file* and writes the output to STDOUT, whereas

```
$ cat file | tr -cd 'aeiou'
```

removes all non-vowels from *file* and writes the output to STDOUT.

A common idiom uses *tr* to remove non-printable characters from a stream using the filter:

```
$ tr -cd '\11\12\40-\176' < inputFile > outputFile
```

Here we delete the complement of the printable characters, listed in octal, in effect, only printing the *printable* characters. Thus we retain only characters with ASCII value 11 [TAB], 12 [NEWLINE] and characters 040 [SPACE] through 0176 [~].

16.4 *cut*

This simple command has two useful variants. In the first form, it cuts out a set of characters specified by index and/or range:

```
$ cut -c1-80 fileName | lp
```

will truncate each line of *fileName* to 80 characters and writes the results to the *lp* command, which will print the truncated lines. Individual characters can be *cut* out as well:

```
$ cut -c1-5,7,9,11-20 fileName
```

cuts out characters 1 through 5, 7, 9 and 11 through 20 for each of *fileName*'s lines.

In the second form of *cut*, fields can be cut out. Additionally, a field delimiter, which defaults to <TAB> can be specified:

```
$ cut -d: -f1,7 /etc/passwd
```

prints the first and seventh fields of each colon delimited record (line) of */etc/passwd*.

Like many filters, *cut* accepts a list of filenames. But, and this is important, if *cut* is invoked *without* parameters, it reads input from STDIN. Hence the *cut* command can be used in a pipeline, as the following example shows:

```
#-----  
# The read command reads from STDIN. Because of the pipe, the  
# read's STDIN is the STDOUT of the previous command. This  
# script thus reads one line at a time from /etc/passwd. From  
# the current line, it cuts out the first and seventh fields  
#  
#-----  
cat /etc/passwd | while read passwdLine  
do  
    echo "$(echo $passwdLine | cut -d: -f1) uses $(echo  
$passwdLine | cut -d: -f7)"  
done
```

16.5grep

grep, which stands for *general regular expression*, is perhaps the most widely used filter. Its syntax:

```
$ grep [options] pattern files
```

filters out only those lines of *files* that match *pattern*. See the section on *regular expressions* for a discussion on pattern matching.

As with many filters, if no files are passed to *grep* on the command line, *grep* operates on its default file, STDIN.

The common options for *grep* are:

- `-v` display unmatched lines
- `-n` display line numbers
- `-c` display a count
- `-i` ignore case
- `-l` list file names
- `-s` suppress error messages
- `-w` search for the regex as a word

Thus:

```
$ vi $(grep -l $(which perl)) *
```

edits all *perl* scripts in the current directory.

16.6awk

awk is a full programming language named after its creators Aho, Weinberger, Kernighan that has a relatively strong C flavor. Variations of the language abound, with *nawk* (new *awk*), and *gawk* (gnu *awk*) being the most common. In what follows, we will assume *awk* is aliased to *gawk*.

We will only cover the basic elements of *awk* deferring a more in depth view to another occasion.

The format for an *awk* command is

```
awk [-Fc] [-f cmdFile | commands] [files]
```

where *commands* is enclosed in quotes (usually single quotes so that variable expansion doesn't occur) and is a sequence of:

```
pattern { ACTION }
```

where *pattern* is usually expressed as a regular expression enclosed in slashes. These commands can either be listed on the command line or can be stored in a file of your choice and invoked using the *-f* option.

awk is a filter that operates on a list of files if specified or defaults to operating on STDIN if no files are specified. Each line of each of the listed files is parsed into tokens delimited by the *awk* variable FS (field separator, which by default is *whitespace*) and stored in the *awk* variables \$1, \$2, \$3 and so on. Subsequently each ACTION is performed on the line if the *pattern* is matched.

If *pattern* isn't specified, then ACTION is indiscriminately applied to each line. On the other hand, if ACTION isn't specified, the default action *print* is invoked for each line. Either a *pattern* or and ACTION or both must be specified.

Thus:

```
$ awk '/marshall/' /etc/passwd
```

invokes the default action – which is *print* – on each line of */etc/passwd* that matches the pattern *marshall* and shows that *awk* with the default action acts like the *grep* command.

On the other hand,

```
$ awk '{print}' myFile
```

prints each line of `myFile` indiscriminately since no pattern was given. Without a pattern, therefore, *awk* acts like the *cat* command.

A slightly more elaborate example

```
$ awk -F: '{printf "%s \t uses shell %s", $1, $7}' /etc/passwd
```

prints out :

```
...
jcporter uses shell /usr/bin/bash
rottmann uses shell /usr/bin/bash
jcpsys uses shell /usr/bin/bash
exp9029 uses shell /bin/ksh
gibbsk uses shell /bin/bash
rgoyal uses shell /bin/bash
```

Here, the `-F:` option sets the *awk* variable `FS` to `:`, so that when each line of `/etc/passwd` is parsed, it uses the `:` as the field separator, with the result that `$1` stores the user name and `$7` stores the shell that user uses.

The keyword `BEGIN` (all caps) indicates an action that occurs before any of the input files are read. It is commonly used for printing a heading or title line for a report before any data is processed and to assign values to built-in variables.

The `END` keyword (all caps) indicates an action occurs after all input records have been read and fully processed. It is commonly used to print summary statements.

Thus

```
$ awk -F: 'BEGIN {print "Bash users\n"}
/bash/ {print $1; ++count}
END {printf "%d bash users in all\n", count}' /etc/passwd
```

prints

```
Bash users

jcporter
rottmann
jcpsys
...
94 bash users in all
```

Noteworthy in this example is the use of the variable `count`. Whenever a line matches the pattern `/bash/`, *count* is incremented by 1. After all lines are processed, the body of the `END` keyword is executed, which prints the footer.

The first time *awk* encounters a reference to a variable, the variable is created and initialized to a null string or 0 if it is used in an arithmetic expression.

For complex *awk* scripts, it is worthwhile to put the script in a file and invoke it with the `-f` option:

```
$ awk -f myAwkScript files
```

or, better, use the she-bang notation to notify the kernel that the script is to be interpreted using *awk*. To this end, the first line of the script should be

```
#!/usr/bin/awk -f
```

the full path to *awk* with the `-f` option.

The process of creating an *awk* script skeleton is automated using the *cres* script with the `-ta` option. This script, which is described in section 17.8, was used to create all of the following examples.

16.6.1 Built in variables

The following variables are built in to *awk*. We will see how some of these are used in the examples section.

NF	- Number of fields in current record
RS	- Input record separator (newline)
FS	- Input field separator (spaces/tabs)
NR	- Number of records from beginning of first input file
OFS	- Output field separator (space)
ORS	- Output record separator (newline)
FNR	- Number of records from beginning of current file
OFMT	- Output format for numbers (%.6g)
ARGC	- Number of command line arguments
ARGV	- Array of command line arguments
FILENAME	- Current input filename
RSTART	- Index into a string, set by match()
RLENGTH	- Length of a substring, set by match()
CONVFMT	- String conversion format for numbers (%.6g)
ENVIRON	- Associative array for environment variables
SUBSEP	- Separator character for array subscripts (\034)

16.6.2 Example Scripts:

In lieu of a more in depth discussion about *awk*, we present a few examples and delegate an in depth examination of the language to another occasion.

16.6.2.1 args

As in other languages, command line arguments made available in an array, as is the total number of them. The *gawk* variable `ARGC` contains the total number of command line arguments, whereas the *gawk* array variable `ARGV` contains them. `ARGV[0]` is the name of the script itself.

Here is a small example. Note the she-bang notation indicates it is a *gawk* script. Note also that the `BEGIN` block needs to have the starting brace on the same line as the word `BEGIN`.

This example also shows the `for()` loop, which is exactly as in C/C++ and Java.

```
#!/opt/sfw/bin/gawk -f
# @(#) Show the use of the built-in variables ARGC and ARGV
#=====
# File           : args
# Description    :
#
# Revision History:
#
#   05 Mar 2003 - marshallb
#   Initial Version.
#=====
BEGIN {
    printf "ARGC = %d\n", ARGC
    for( i = 0; i <= ARGC; ++i )
        printf "Argument %d is %s\n", i, ARGV[i]
}
```

16.6.2.2 countBash

This simple script shows a few things. First off, the field separator is set to `:` in anticipation of parsing the `/etc/passwd` file, in which the fields are colon-separated. Secondly, note that the `if/else` construct is exactly as in C/C++ and Java. What is different is the condition of the *if* statement. Here we have an example of the *match operator*. It returns true if the seventh field of the current line matches the pattern `'bash'`.

The opposite of the `'~'` operator is the `'!~'` – doesn't match – operator.

```

#!/usr/bin/awk -f
# @(#) Example of conditional statement in awk
#=====
# File           : countBash
# Usage          : countBash /etc/passwd
#
# Revision History:
#
#   05 Mar 2003 - marshallb
#       Initial Version.
#=====
#-----
# Set the field separator to ':'. Note this is equivalent to invoking awk
# with the -F: option.
#-----
BEGIN { FS = ":" }
{
    #-----
    # An example of the match operator ~. It says if the seventh field
    # matches the pattern bash, then ...
    #-----
    if( $7 ~ /bash/ )
    {
        printf "%s uses %s\n", $1, $7
        ++bCount
    }
    else
    {
        printf "%s DOES NOT use %s\n", $1, $7
        ++nbCount
    }
}

#-----
# Print a footer with the total number of bash users
#-----
END {
    printf "There are %d bash users on the system\n", bCount
    printf "There are %d NON bash users on the system\n", nbCount
}

```

16.6.2.3 countWords

This example uses *associative arrays*. These are arrays in which the index doesn't have to be an integer. It uses a number of *awk* builtin variables, which are listed in section 16.6.1. Additionally, it shows how to access array elements and how to walk an array.

```

#!/usr/bin/awk -f
# @(#) Example of using an associative array
#=====
# File           : countWords
# Description     :
#
# Revision History:
#
#   05 Mar 2003 - marshallb
#       Initial Version.
#=====
BEGIN {
    print "Starting countWords"
}

{
    #-----
    # NR  - current record number
    # NF  - the number of fields on the current line
    # $NF - the value of the last field
    #-----
    print "Processing line ", NR
    for (i = 1; i <= NF; ++i)
    {
        print "Storing away", $i
        #-----
        # $i is the i-th word on the line. Store in the associative array,
        # a, and increment its count (The value of a[$i] is initialized to 0
        # on construction.
        #-----
        a[$i]++
        print $i, "is stored with count", a[$i]
    }
}

END {
    #-----
    # FILENAME - awk variable containing the name of the current file awk
    # is processing.
    #-----
    printf "The breakdown of the words in %s are:\n", FILENAME
    #-----
    # Here is how you walk an array
    #-----
    for (c in a) { print c, a[c]}
}

```

16.6.2.4 readFile

This small script gives an example of how to read from a file. We first check if a command line argument is specified. If not, ARGV is one (because the name of the script

is stored in ARGV[0], hence ARGV is always at least one) so we print a usage message and exit. It uses the *getline* function to read one line at a time from the passed in file.

```
#!/opt/sfw/bin/gawk -f:w
# @(#) Example of reading from a file in gawk. Here print passed in file.
#=====
# File           : fileT
# Description    :
#
# Revision History:
#
#   05 Mar 2003 - marshallb
#       Initial Version.
#=====
BEGIN {
    if( ARGV == 1 )
    {
        printf "Usage: %s <filename>\n", ARGV[0]
        exit
    }
    fileName = ARGV[1]

    #-----
    # use the getline function to read a line from the file filename into
    # the variable line.
    #-----
    while( (getline line < fileName) > 0 )
        printf "---> %s\n", line
}
```

16.6.2.5 functionTest

This example shows how to define a user defined function. Such a function can be defined anywhere within the script. Additionally, it can be called from anywhere in the script, including the BEGIN and END blocks.

```
#!/opt/sfw/bin/gawk -f
# @(#) Example of user defined function in gawk
#=====
# File           : functionT
# Description    :
#
# Revision History:
#
#   05 Mar 2003 - marshallb
#       Initial Version.
#=====
#-----
# User defined function
#-----
function myFunc(prefix, data)
```

```
{
    printf "%s %s\n", prefix, data
}

BEGIN {
    #-----
    # Note the function can be used inside of a BEGIN block
    #-----
    myFunc("BEGIN", "A function can be called from within the BEGIN block")
}

{
    #-----
    # $0 - the current line
    # sprintf() produces a string consisting of the current record (line)
    # number followed by a space followed by the line itself.
    #-----
    myFunc("MAIN", sprintf(" %d %s", NR, $0))
}

END {
    myFunc("END", "A function can be called from within the END block")
}
```

16.7 *sed*

sed, the stream editor, is a programming language in its own right. We reserve its use, to simple editing in this document, and will present a more intense view of the language in another setting.

sed, however, is also a filter and in this context it is used to perform editing on the output of the previous command. Thus a pattern can be *substituted* for another, lines can be *deleted*, text can be *inserted* above or *appended* below lines that match a pattern. We shall take a look at each of these cases.

sed can be called as follows:

```
$ sed -e 'sedCommands' -e 'moreSedCommands' files
```

where *sedCommands* is a semi-colon or newline separated list of *sed* commands, or as:

```
$ sed -f sedCommandsFile files
```

if all the *sed* commands are persisted to the file *sedCommandsFile*, or as a standalone executable by using the *she-bang* notation to tell the kernel that the given script is a *sed* script. In this case, the first line of the script must be:

```
#!/usr/bin/sed -f          # full path to sed
```

or

```
#!/usr/bin/sed -nf        # full path to sed
```

if default printing of each processed line is to be suppressed.

In any case *sed* operates on all the files listed on the command line or on STDIN if none are present. *It does this by applying, in order, each of the sed commands to each of the lines of each of the files.*

sed operates on each file and writes output to STDOUT leaving the original files untouched, by default. However, the output can easily be captured by redirection or by using the *Unix tee* command as we'll see shortly.

The trivial *sed* script, consisting of a single empty command, acts as the *cat* command:

```
$ sed '' fileName
```

```
outputs fileName to STDOUT, just like cat
```

That is because by default every line processed by `sed` is output to STDOUT.

The `sed` command 'p' prints out the so called *pattern space*, which is the current line `sed` is processing possibly modified in some way by one of `sed`'s commands. Thus

```
$ sed 'p' fileName
```

```
    duplicates each line of fileName to STDOUT
```

Now, the '-n' command switch notifies `sed` **not** to print out the *pattern space* **unless** explicitly told to do so using the 'p' command. Hence

```
$ sed -n 'p' fileName
```

```
    outputs fileName to STDOUT, just like cat
```

puts us back to square zero.

But we actually have an advantage since *each sed command can be qualified by an address*, shown below so that

```
$ sed -n '3p' fileName
```

```
    outputs the third line of fileName to STDOUT
```

and

```
$ sed -n '/AT&T/p' fileName
```

```
    outputs only those lines of fileName that contain  
    the pattern 'AT&T' to STDOUT
```

Better yet,

```
$ sed -n '3,5p' fileName
```

```
    outputs lines 3 - 5 (inclusive) of fileName to  
    STDOUT
```

and

```
$ sed -n '/startPattern/, /endPattern/p' fileName
```

```
    outputs every line of fileName between the lines  
    containing startPattern and endPattern inclusive
```

Finally, to exhaust the concept of a *sed line address*, we see that line numbers and patterns can be used in conjunction:

```
$ sed -n '/startPattern/, 15p' fileName
```

```
outputs every line of fileName between the lines
containing startPattern and line number 15
inclusive
```

It turns out that a line address is optional with any *sed* command. It can be a pattern described as a regular expression surrounded by slashes, a line number, or a line-addressing symbol. Most *sed* commands can accept two comma-separated addresses that indicate a range of lines.

sed commands can be grouped at the same address by placing the commands between braces, the last of which must be on a line by itself:

```
$ sed 'address {
      = # print the line number
      p # print the line
    }' fileName
```

16.7.1 line addresses

We have seen that *sed* commands can be prefixed by an optional address. Here we briefly rehash what constitutes a *sed* address.

- A single number is an address
- The special symbol '\$' is an address
- A regular expression between forward slashes is an address
- A range consisting of any combination of the above mentioned addresses is an address

Thus

```
/start/, /end/
```

indicates all lines between any line containing the pattern *start* and the pattern *end* or the end of the file if that pattern is not found, inclusive.

Similarly

```
/start/, $
```

indicates all lines between the first line containing the pattern *start* and the end of the file.

While the address consisting of a sole number specifies to *sed* that it should only issue commands for that line number on the files passed to *sed*, an address consisting of a

single pattern dictates that *sed* should operate on any line in any of the passed in files that contain that pattern.

Most *sed* commands can operate on any address, but some operate only on non-range addresses.

16.7.2 substitute

The simplest and perhaps most commonly used *sed* programs uses the *substitute* command:

```
$ unixCommand | sed 's/this/that/'
```

takes the output of *unixCommand* and replaces the *first* occurrence of the *this* regular expression with *that*, whereas

```
$ unixCommand | sed 's/this/that/g'
```

takes the output of *unixCommand* and replaces *every* occurrence of the *this* pattern with *that* and

```
$ unixCommand | sed 's/this/that/2'
```

takes the output of *unixCommand* and replaces the *second* occurrence of the *this* pattern with *that*.

As an example, if

```
$ echo $fileName
```

```
info2nc3.cpp
```

then

```
$ echo $fileName | sed 's/\.*$//' # strips the extension
```

```
info2nc3
```

strips the extension. Here the pattern `\.*` matches the literal `.`, since it is *escaped*, followed by any character (`.`) any number of times (`*`). See the section on *regular expressions* in this document for details on *patterns*.

Building on this example, the following script replaces each file name with one extension with the file name with another extension:

```

#!/usr/bin/bash
#=====
# File           : replaceExtension
# Description    :
#
# Revision History:
#
#   14 Feb 2003 -
#       Initial Version.
#=====
origExt=$1
newExt=${2:?Usage: $0 <oldFileExtension> <newFileExtension>}

#-----
# For each file with the original extension ...
#-----
for f in *.$origExt
do
    #-----
    # ... strip the extension and add the new one
    #-----
    mv $f $(echo $f | sed 's/\..*//').$newExt
done

```

As another simple example, suppose we have a file in which we would like every occurrence of ‘IBM’ to be replaced with ‘International Business Machines’. The two steps accomplish this:

```

$ sed 's|IBM|International Business Machines|g' fileName > fileName.$$
$ mv fileName.$$ fileName

```

Note that:

```

$ sed 's|IBM|International Business Machines|g' fileName > fileName

```

wouldn’t work since input redirection occurs before *sed* is executed truncating *fileName* before *sed* even has a chance of reading it. See Appendix C for the shell’s parsing sequence. Note the use of the variable ‘\$’, which contains the process’ PID to create a temporary file. Furthermore notice that the pipe symbol, ‘|’, is used as a pattern separator instead of the usual slash. This is often used if the pattern itself has a slash. Any character, with the exception white space, can be used a pattern separator.

Recall that *sed* commands can be qualified by a *line-address*:

```

$ sed -n '25,30s|IBM|International Business Machines|gp' fileName

```

outputs only those lines between lines 25 and 30 of *fileName* which contained the pattern ‘IBM’, but with ‘International Business Machines’ in its place.

Any line-addressing scheme can be used in conjunction with the *substitute* command.

Note, in this last example, the use of the `-n` option to suppress printing of each line and the `sed 'p'` command after the `'g'` (for global), to explicitly print lines of interest.

The commands of the last *sed* invocation can be stored in a file and then be invoked by *sed*:

```
$ cat sed1
```

```
#n                                # equivalent to specifying -n on the command line
25,30 {
    s/IBM/International Business Machines/gp
}
```

The *sed* commands are then executed thusly:

```
$ sed -f sed1 fileName
```

with the same results as above.

16.7.3 delete

The *delete* command clears the *pattern space* with the result that no further *sed* commands operate on that line.

Thus

```
$ sed 'd' fileName
```

ostensibly does nothing, whereas

```
$ sed '3d' fileName
```

prints to STDOUT all lines of *fileName* with the exception of line 3.

As one would expect, *line-addresses* can qualify the *delete* command.

16.7.4 insert, append and change

These commands modify the *pattern space* by inserting above or appending below or just changing the *pattern space*. Their syntax is as follows:

```
[line-address] i\  
text \
```

```
more text

[line-address] a\
text \
more text

[line-address] c\
text \
more text
```

Each command must be on a line followed by the back-slash character with the text that modifies the *pattern space* below it. It is a common mistake to have white space after the backslash.

Thus the command:

```
$ sed 'a\
    NEW TEXT 1 \
    NEW TEXT 2' fileName
```

will print out each line of *fileName* followed by the two lines of NEW TEXT.

The result of the command:

```
$ sed 'i\
    NEW TEXT 1 \
    NEW TEXT 2' fileName
```

on the other hand, will print out, for each line of *fileName* the two lines of NEW TEXT followed by a line of *fileName*.

Finally, the command

```
$ sed 'c\
    NEW TEXT 1 \
    NEW TEXT 2' fileName
```

will totally replace the *pattern space* and print out the two lines of NEW TEXT for each line of *fileName*, without printing out a single line of that file.

As with all *sed* commands, the three commands introduced in this section can be qualified by a *line-address*. In the true spirit of *Unix*,

16.7.5 read

The *read* command takes a single parameter: the name of a file. It appends the contents of the file to the *pattern space*.

Therefore:

```
$ cat separator
```

```
-----
```

```
$ sed 'r separator' fileName
```

will print each line of *fileName* followed by 50 dashes.

By qualifying the *sed* command with a meaningful address, we can be very creative. Suppose, for example, that there are a number of C++ files that each need an *AT&T* header with copyright information and such. Then if the header is stored in the file *attHeader*:

```
$ sed -i.bak '1r attHeader' *.cpp      # append the contents of the header after line 1
```

will append the header after the first line, which is not what we would like. Note that the command switch *-i .bak* does an *inplace* modification and saves off the original file with a *.bak* extension.

```
$ for file in *.bak
do
    echo "moving $file back to ${file%.bak}"
    mv $file ${file%.bak}
done
```

restores each C++ file. The following combination of *sed* with *command substitution* will do the trick:

```
$ sed -i "1i\
$(cat attHeader)" *.cpp
```

since the contents of the output of the *cat* command will be inserted before line 1. Double quotes were necessary since *command substitution* will not occur between single quotes. The *-i* switch didn't have an extension, which signifies the changes happen *inplace* with no backup. Clearly, only use this switch if you are sure of the outcome of the command.

16.7.6 write

The *sed* *write* command complements the *read* command. It takes a single argument, the name of a file to which to write the *pattern space*.

```
$ cat formatProp
```

```
#n
s/\(.*\)=\(.*\) /The key \1 has value \2/pw props
```

```
$ cat myProperties
```

```
startdir = ~marshall/myProg
propFile = ~marshall/myProg/properties
...
```

Then

```
$ sed -f formatProp myProperties
```

produces the file *props* with contents

```
The key startdir has value ~marshall/myProg
The key propFile has value ~marshall/myProg/properties
```

In this case, the *sed* commands were put in a file, from which they can be invoked by using the *-f* command line switch.

The leading ‘#n’ in the file is equivalent to specifying *-n* on the command line and is used to suppress the default printing of the *pattern space* after reading and processing each line of every file *sed* operates on.

The second line contains an example of using *backreferences*. These are patterns expressed using regular expressions enclosed in *escaped parentheses*. The first matched pattern is stored in the *sed* special variable \1, the second in \2 and so on. The *substitute* command then replaces the pattern space with the string ‘The key \1 has value \2’ with appropriate values substituted in place of the special variables *and* simultaneously prints the pattern space to STDOUT because of the ‘*p*’ command *and* writes the pattern space to the file *props* as dictated by the ‘*w*’ command.

16.7.7 Example

It is helpful to see an example of a *sed* script using all of the above described elements. Although this example is contrived, it serves well as a stepping stone.

```
$ sed '1,50 -n {
    1,10p                # print the first 10 lines
    /start/, /end/ {     # in the range between start and end
```

```
/comment/d          # remove any line with the word comment  
/TODO: add header/r header # add the contents of the header file  
}  
/important/p        # print any line that contains important  
}' files
```

The gist is that the above elements can be combined in any reasonable way. Note how braces are used to group commands together.

17 Sample scripts

We present a few scripts that can be used as the starting point for more elaborate ones.

17.1 *xtr*

In the article “Unix Time Sharing System Forward” (The Bell System Technical Journal, July-Aug 1978), Doug McIlroy et al purport the following principles:

- make each program do one thing well
- expect the output of every program to become the input of another

It is thus not surprising to find Unix utilities that don’t have options that one might expect. As an example, the *tar* utility doesn’t have the built in facility of extracting files of your choice.

The following simple script shows how easy it is to expand *tar*’s functionality:

```
#!/usr/bin/bash
# @(#) wraps tar xvf to extract the files matching passed in regex
#=====
# File           : xtr (extract)
# Description    : extract from a tar file all files that match a regex
#
# Revision History:
#
#   26 Feb 2003 - marshallb
#       Initial Version.
#=====
tarFile=${1:?Usage: $0 <tarFile> <pattern>}
patt=${2:?Usage: $0 <tarFile> <pattern>}
#-----
# tar tvf $tarFile gives a long listing of the files the tarball contains.
# The results are piped to awk to extract the last field of each line,
# which is the file name. Run these filenames through grep to filter out
# only those that match the passed in pattern. Finally, because of
# command substitution, these filtered files are passed to tar xvf ... to
# extract the files on interest.
#-----
tar xvf $tarFile $(tar tvf $tarFile | awk '{print $NF}' | grep $patt)
```

17.2xt

This script creates a number of xterms specified by the user with semi-random colorschemes. Optionally, a user can specify a title for each xterm.

It shows the use of arrays in bash.

```
#!/bin/bash
#####
# File      : xt
# Description : Create a number of xterms with semi-random color schemes.
#             Do this by creating an array of color schemes of your fancy
#             and randomly choosing an element of this array.
#
# Revision History:
#
# 20 Jan 2001 - marshallb
#   Initial Version.
#####
# Function:      isNumeric
# Description: This function returns false if the passed in argument is NOT
#              numeric.
#=====
function isNumeric
{
    expr $1 + 1 > /dev/null 2>&1
}

#=====
#                                     MAIN
#=====
#-----
# Define the colorschemes. Format: fgColor bgColor cursorColor.
# If the cursorColor is not specified, it defaults to black.
# The syntax declare -a varName defines varName as an array.
#-----
declare -a colorscheme
colorscheme=("NavajoWhite DarkOrchid" "DarkOrchid NavajoWhite" "NavajoWhite
RoyalBlue" "RoyalBlue NavajoWhite" "NavajoWhite blue4" "blue4 NavajoWhite"
"grey30 LawnGreen" "LawnGreen grey30" "grey35 LightCyan" "LightCyan
grey35" "AntiqueWhite MidnightBlue" "MidnightBlue AntiqueWhite yellow"
"LightSkyBlue4 seashell1" "seashell1 LightSkyBlue4" "aquamarine4 seashell1"
"seashell1 aquamarine4" "LightSalmon blue3" "blue3 LightSalmon" "ivory4
CadetBlue1" "CadetBlue1 ivory4" "grey35 wheat1" "wheat1 grey35" "maroon
NavajoWhite1" "NavajoWhite1 maroon" "maroon yellow" "yellow maroon"
"DarkBlue yellow" "grey79 DarkBlue" "grey79 DarkBlue" "DarkBlue grey79"
"azure1 DodgerBlue1" "DodgerBlue1 azure1" "DodgerBlue1 azure1" "azure1
DodgerBlue1" "grey85 gray22" "gray22 grey85" "wheat4 yellow1" "yellow1
wheat4" "NavajoWhite1 maroon" "maroon NavajoWhite1" "DeepSkyBlue
MediumPurple4")

#-----
# If the first cmd line param is an integer, then create that many xterms.
```

```

# In this case, the xterm's title will be the colorscheme.
# Otherwise, the first command line arg is the xterm's title (use single
# quotes if the title has spaces) and the number of xterms is either the
# second command line arg, if it exists, or 1
#-----
case $# in
  0)
    num=1 ;;
  *)
    if isNumeric $1 ; then
      num=$1
    else
      title="$1"
      num=${2:-1}
    fi ;;
esac

for(( i = 0; i < $num; ++i )); do
  #-----
  # Choose a random number between 0 and the number of colorschemes minus
  # 1. Use the enviromental variable RANDOM to return a random integer.
  #-----
  choice=$(( RANDOM % ${#colorscheme[*]}))
  #-----
  # Now randomly choose the foreground field (fgf) to be 0 or 1. The
  # background field will be the other field. The cursor color is the
  # third field, if it exists, otherwise 'black'
  #-----
  fgf=$(( RANDOM % 2 + 1))
  bgf=$(( fgf % 2 + 1))
  fg=$(echo ${colorscheme[$choice]} | cut -d' ' -f$fgf)
  bg=$(echo ${colorscheme[$choice]} | cut -d' ' -f$bgf)
  cr=$(echo ${colorscheme[$choice]} | cut -d' ' -f3)
  : ${cr:=black}
  if [[ -z $title ]]; then
    title="$fg on $bg"
  fi
  xterm -title "$title" -ls -sb -sl 10000 -fn fixed -fg $fg -bg $bg -cr
$cr&
done

```

17.3tree

The following script visually displays all files in and below the passed in directory. It uses the *Unix find* command in conjunction with *sed*.

```
#!/bin/bash
# @(#) tree v1.0 Visual display of a file tree
#=====
# File           : tree
#
# Revision History:
#
#   25 Aug 2001 - Brian P. Marshall
#   Initial Version.
#=====
find ${1:-.} -print | sed '
{
  ld          # remove the first line (the passed in directory)
  s![^/]*!/    | !g  # replace every occurrence of '/' with 3 spaces
  s!^ *|!!     # remove leading white space
}'
```

17.4 fileManager

The following example uses the *getopts* builtin. This is used to parse command line arguments (see man pages for details).

```
#!/usr/bin/bash
#####
# File      : getoptEG
# Description : getopt is a command that allows us to process
#              command line arguments.  getopt takes a string
#              of legal command line options and a variable
#              (option in the following example).
#              getopt is processed in a loop.  A leading ':' in
#              the command line string indicates that option will
#              be set to '?' if an invalid option is entered at
#              the command line.
#              A ':' following an option means that the option expects a
#              value.  If a value is not found, option is set to ':', if
#              it is, it is stored in the variable OPTARG.
#
# Revision History:
#
# 06 Mar 2001 - marshallb
#   Initial Version.
#####
#-----
# The following getopt clarifies what command line options this script
# accepts.  The option -f expects an argument as manifested by the trailing
# :.  All other options - a, b, c, g, q - do not expect arguments.
# In the case of the -f option, its argument is contained in the variable
# $OPTARG.  OPTIND, on the other hand, is the index of the current command
# line argument getopt is processing.
#-----
while getopt :abcf:gq option; do
    echo "Processed option $option"
    case $option in
        \?) echo "***invalid option letter == $OPTARG";;
        :) echo "Missing parameter $OPTARG";;
        a) echo "The -a switch was found"
            echo "OPTIND is $OPTARG";;
        +a) echo "This turns OFF switch a"
            echo "OPTIND is $OPTARG";;
        b) echo "The -b switch was found";;
        c) echo "The -c switch was found";;
        +c) echo "This turns OFF switch c";;
        f) echo "The -f switch was found"
            echo "argument to f is: \"$OPTARG\""
            echo "OPTIND is $OPTARG";;
        g) echo "The -g switch was found";;
        q) echo "The -q switch was found"
            echo "Will now QUIT this script!"
            echo "OPTIND is $OPTARG"
            break;;
    esac
done
```

```
done

#-----
# Since OPTIND is the index of the current command line argument
# to be processed, we can process the remaining arguments by shifting
# $OPTIND times.
#-----
shift $(( OPTIND - 1 ))

if(( $# > 0 )) ; then
    typeset i i=1

    echo "Remaining command line arguments: "
    #-----
    # Note the syntax: for arg    is equivalent to    for arg in $*
    #-----
    for arg
    do
        echo "$i. $arg"
        (( i += 1 ))
    done
fi
```

17.5 getNC3Schema

This script shows how to retrieve data from a DB2 database.

```
#!/usr/bin/env bash
# @(#) Get the table definition for each table in NC3
#=====
# File           : getNC3Schema
# Description    : Writes to the file nc3Schema, the table definition of
#                  each table in nc3's schema
#
# Revision History:
#
#   08 Jan 2003 - marshallb
#   Initial Version.
#=====
#-----
# Remove the nc3Schema file and don't ask if I really want to (-f).  If
# there are any errors, don't report them.  This could happen if the file
# doesn't exist, for example.
#-----
rm -f nc3Schema 2>/dev/null

#-----
# Connect to the db2 database. Supply real hostName, userName, passWord.
# The established connection is used to retrieve table descriptions in the
# body of the for loop.
#-----
db2 "connect to hostName user userName using passWord"

#-----
# Note that we create another db2 connection in the command substitution of
# the for loop, which retrieves a list of the tables in NC3's schema.  This
# is necessary since this command executes in a subshell.
#-----
for table in $(db2 "connect to hostName user userName using passWord"
>/dev/null
2>&1; db2 "list tables for schema nc3" | sed -n '4,$p' | egrep -v '^$|^ '
| awk
'{print $1}')
do
    echo "Getting $table's schema"
    echo "          ===== $table =====" >>
nc3Schema
    db2 "describe table nc3.$table" | grep -v 'record(s) selected' >>
nc3Schema
done
```

In this example, we need to connect to db2 twice. The reason is that the command to retrieve all table names in NC3's schema executes in a subshell and isn't be able to use the connection established in the parent to describe each table.

17.6 ftpFiles

This script shows how to retrieve data from a remote host via *ftp*. It can be tailored easily for specific needs.

```
#!/usr/bin/bash
#####
# File           : ftpFiles
# Description    :
#
# Revision History:
#
#   13 Feb 2003 -
#       Initial Version.
#####

#=====
# Function: usage
#=====
function usage
{
    cat <<EOF
Usage: $0 -h <hostName> -u <userName> -p <passWord> -c <cmdsFile>
EOF

    exit
}

#=====
# Function: getFileNames
#=====
function getFileNames
{
    ftp -n <<-EOT
connect $hostName
user $userName $passWord
ls
EOT
}

#=====
#                               MAIN
#=====

#-----
# Use the getopts builtin to retrieve command line arguments.  Valid
# options are: -h <hostName> -u <userName> -p <passWord>.  The colons
# following each option indicate that those options expect an argument.
# getopts stores that argument in OPTARG.
#-----
while getopts :h:u:p: option
do
    case $option in
        h) hostName=$OPTARG ;;
```

```

        u) userName=$OPTARG ;;
        p) passWord=$OPTARG ;;
        c) cmdsFile=$OPTARG ;;
        \?) usage ;;
    esac
done

#-----
# If the hostName or userName isn't specified on the command line, choose
# the appropriate environmental variable, presumably set in your login
# script. If they are not defined, prompt for them. Note the leading ':'
# - the NOOP operator
#-----
: ${hostName:=${FTPHOST:-$(read -p "Enter host: "; echo $REPLY)}}
: ${userName:=${FTPUSER:-$(read -p "Enter user: "; echo $REPLY)}}
: ${passWord:=$(read -p "Enter password: " response; echo $response)}

#-----
# SOURCE_DIR is presumably an environmental variable defined in your login
# script. If it is not defined, choose the current directory, '.',
# instead.
#-----
cd ${SOURCE_DIR:-.}

#-----
# If there are no files in the current directory ...
#-----
if (( $(ls | wc -l) == 0 ))
then
    files=$(getFileNames)
    echo "Pulling data files from $hostName"
    #-----
    # ... so fetch the files.
    #-----
    # ftp -nv $hostName <<-EOT 2>> $FTPLOGFILE
    ftp -nv $hostName <<-EOT
    user $userName $passWord
    binary
    hash
    $(for file in $files
    do
        echo get $file
    done)
    quit
    EOT
fi

echo "Ftp transfer from $hostName complete"

```

setupSSH

This script takes a number of hosts as parameters. Once it is run, a user will be able to login to those remote hosts using *ssh* or copy files to and from those remote hosts using *scp* without having to supply a password. This simplifies executing scripts on remote hosts.

```
#!/ env bash
# @(#) Usage: setupSSH hostList
#=====
# File      : setupSSH
# Description : This script takes a list of hostnames as parameters.
#           It:
#           o creates the directory ~/.ssh if it doesn't exist
#           o creates the directory ~/.ssh on each remote host if
#             it doesn't exist
#           o pushes the public keys to each remote host
#
# Revision History:
#
# 03 Mar 2003 - bmarshal
# Initial Version.
#=====
#-----
# If no command line arguments were specified, prompt the user for a list
# of hosts
#-----
if (( $# == 0 ))
then
    read -p "Enter a space separated list of hosts to which you use ssh: " hosts
else
    hosts=$*
fi

#-----
# Bail if there is no list of hosts
#-----
if [[ -z $hosts ]]
then
    echo "No hosts to specified. Exiting ..."
    exit 1
fi

#-----
# Create the .ssh directory in $HOME, if it doesn't already exist. If the
```

```

# directory does exist, remove all public keys
#-----
[[ ! -d ~/.ssh ]] && mkdir .ssh
cd ~/.ssh

#-----
# Generate the public keys if they are not already present
#-----
for type in dsa rsa rsa1
do
    if [[ $type == [dr]sa ]]
    then
        file=id_$type
    else
        file=identity
    fi
    if [[ ! -e ${file}.pub ]]
    then
        echo "Generating public key for $file"
        echo ssh-keygen -t $type -P " -f $file
        ssh-keygen -t $type -f $file -P "
    else
        echo "Using ${file}.pub in ~/.ssh"
    fi
done
#-----
# Create the .ssh directory on the remote machine. Then append the public
# keys on your box to the ~/.ssh/authorized_keys file
#-----
for host in $hosts
do
    echo "$(tput bold)creating .ssh directory on $host$(tput rmso)"
    user=${userName:-$(id -un)}
    read -p "Enter your user name on $host [$user]: " userName
    : ${userName:= $user}

    #-----
    # Use ssh to
    # 1. Create the directory .ssh on the remote machine in $userNames
    #    home directory. The if-statement will fail if the user's shell
    #    is not Korn or Bash, in which case it will always try to create
    #    the directory
    # 2. Concatenate the public keys to the authorized_keys file in the
    #    .ssh directory.
    #-----
    cat *.pub | ssh $userName@$host 'if [[ ! -d .ssh ]]

```

```
        then
            mkdir .ssh
        fi
        cd .ssh
        cat >> authorized_keys
        echo created .ssh on $host
        ls -ltr'
done
```

cres

The *cres* (create script) script takes optional command line arguments `-t [abkps]` and creates the skeleton of a script. It does this by adding a *she-bang* line, a header and `chmod`'ing the

```
#!/bin/bash
#=====
# File      : cres
# Description : Create script.  Prompt for a filename if one isn't specified
#              on the command line.  Check if the file exists.  Add the basic
#              lines to the script and then make it executable
#
# Revision History:
#
#   10 Aug 1999 - marshallb
#       Initial Version.
#=====
#-----
# Includes
#-----
. ~/bin/functions/getKey
. ~/bin/functions/lower

#-----
# What kind of script hull shall we create?  This can be specified by using
# the -t option (-t a for awk, -t b for bash and so on).  Default: bash
#-----
getopts :t: cmdLine
case "$cmdLine" in
    t)
        case $OPTARG in
            a*) prgm="$(which awk) -f" ;;
            b*) prgm="$(which bash)" ;;
            k*) prgm="$(which ksh)" ;;
            p*) prgm="$(which perl)" ;;
            s*) prgm="$(which sed) -nf" ;;
            ?) echo "Usage: $(basename $0) [-t <abkps>]"; exit ;;
        esac
        ;;
    ?) prgm="$(which bash)" ;;
esac

#-----
# Shift away the -t and option
#-----
shift $(( OPTIND - 1 ))

#-----
# Get the filename either from the command line if specified or after
# prompting the user for it.  Here's how this works: If a command line arg
# is specified, it is stored in $1.  If this is defined, assign it to
# fileName.  Otherwise, prompt the user for a fileName (read -p "prompt"),
# read the user's input into resp and return it.
```

```

#-----
fileName=${1:-$(read -p "Enter fileName: " resp; echo $resp)}

#-----
# Loop until fileName is not empty (-n) and (-a or &&) a file by that name
# does not exist (! -e)
#-----
until [[ -n "$fileName" && ! -e "$fileName" ]]
do
    #-----
    # If fileName is empty (-z for zero) the re-prompt the user and start
    # at the top of the loop again (continue)
    #-----
    if [[ -z "$fileName" ]]; then
        read -p "Enter fileName: " fileName
        continue
    fi
    #-----
    # If we get here, the file exists, so ask the user if it should be
    # overwritten. Here the -n argument specifies 'no new line'.
    # getKey is a function which retrieves 1 character from the keyboard. -l
    # specifies to convert it to lower case (-u upper).
    # If the user types 'y' or 'Y', break out of the until loop...
    # Note: the [[ ]] syntax is exclusively Korn Shell. In the above if
    # statement single brackets are used ([ ]), which is Bourne shell syntax
    #-----
    echo -n "$fileName exists. Overwrite (y/n)? "
    if [[ $(getKey -l) == 'y' ]] ; then
        break
    fi
    #-----
    # ... otherwise prompt the user for another file name
    #-----
    echo
    read -p "Enter fileName: " fileName
done

#-----
# Get the user's name, which is field 5 of the /etc/passwd (GCOS). Do
# this by grep'ing for the user's id (id -un on Linux) or login name (LOGNAME)
# in the /etc/passwd file and then cutting out the fifth colon delimited field
# Linux: userName=$(grep $(id -un) /etc/passwd | cut -d: -f5)
#-----
userName=$(grep "^$LOGNAME:" /etc/passwd | cut -d: -f5)
read -p "Comment []: " comment

#-----
# An example of a 'here' document. It says cat (write) to $fileName every
# thing after the marker EOF and before that same marker. The '-' (<<-)
# says that leading tabs are allowed and will be stripped before writing
# the here document to the file
#-----
cat > $fileName <<-EOF
    #! $prgm
    # @(#) $comment
    #=====
    # File                : $fileName

```

```
# Description      :
#
# Revision History:
#
#   $(date +"%d %h %Y") - $userName
#   Initial Version.
#=====

EOF

#-----
# Make the file executable (or chmod u+rx,go-w,go+rx $fileName)
#-----
chmod 0755 $fileName

#-----
# 'vi +11 fileName' means vi fileName and place the cursor on line 11.
# We want to place the cursor on the last line of fileName so that the user
# can immediately start editing the script. Therefore first calculate the
# number of lines in the file: line=$(wc -l $fileName | awk '{print $1}').
# Then overlay vi : exec vi +$line $fileName. These two steps can be
# combined:
#-----
exec $EDITOR +$(wc -l $fileName | awk '{print $1}') $fileName
```

17.7 crech

This script accepts a file name and creates a header file containing the definition of a class - of the name of the file's basename. Some basic functionality is added, and more can be added by tailoring this script. In addition, this script creates a cpp file with an empty main() function.

```
#!/bin/bash
#=====
# File      : crech
# Description : Create cpp and hpp files.  Prompt for a filename if one
#             isn't specified on the command line.  Check if the file
#             exists.
#
# Revision History:
#
#   10 Aug 1999 - marshallb
#       Initial Version.
#=====

#-----
# Get the classname either from the command line if specified or after
# prompting the user for.  In either case, strip the extension if there is
# one
#-----
fileName=${1:-$(read -p "Enter fileName: " resp; echo $resp)}

#-----
# Loop until fileName is not empty (-n) and (-a) a file by that name does
# not exist (! -e)
#-----
until [[ -n "$fileName" && ! -e "$fileName" ]]
do
    #-----
    # If fileName is empty (-z for zero) the re-prompt the user and start
    # at the top of the loop again (continue)
    #-----
    if [[ -z $fileName ]]; then
        read -p "Enter fileName: " fileName
        continue
    fi
    #-----
    # If we get here, the file exists, so ask the user if it should be
    # overwritten. Here the -n argument specifies 'no new line'.
    # getKey is a function which retrieves 1 character from the keyboard. -l
    # specifies to convert it to lower case (-u upper).
    # If the user types 'y' or 'Y', break out of the until loop...
    # Note: the [[ ]] syntax is exclusively Korn Shell. In the above if
    # statement single brackets are used ([ ]), which is Bourne shell syntax
```

```

#-----
echo -n "$fileName exists. Overwrite (y/n)? "
if [[ $(getKey -l) == 'y' ]] ; then
    break
fi
#-----
# ... otherwise prompt the user for another file name
#-----

read -p "\nEnter fileName: " fileName
done

className=${fileName%%.*}
hFileName=${className}.hpp
cFileName=${className}.cpp

#-----
# Get the user's name, which is field 5 of the /etc/passwd (GCOS). Do
# this by grep'ing for the user's id (id -un on Linux) or login name
# (LOGNAME)
# in the /etc/passwd file and then cutting out the fifth colon delimited
# field
# Linux: userName=$(grep $(id -un) /etc/passwd | cut -d: -f5)
#-----
userName=$(grep ^$LOGNAME: /etc/passwd | cut -d: -f5)
header=$(upper $(echo $fileName | sed 's/\./_/'))

#-----
# An example of a 'here' document. It says cat (write) to $fileName every
# thing after the marker EOF and before that same marker. The '-' (<<-)
# says that leading tabs are allowed and will be stripped before writing
# the here document to the file
#-----
cat > $hFileName <<-EOF
/*=====*/
|
| File:          $hFileName
|
| Author:        $userName
|
| Date:          $(date +"%d %h %Y")
|
| Description:
|
+=====*/
#ifndef $header
#define $header

#include <iostream>

class $className
{
public: // methods
    $className()
    {
        cout << "$className object " << this << " constructed" << endl;
    }
    $className(const $className& lhs)

```

```

    {
        cout << "$className object " << this << " copy constructed" << endl;
    }
    virtual ~$className()
    {
        cout << "$className object " << this << " destructed" << endl;
    }

    $className& operator=(const $className& rhs)
    {
        return *this;
    }
    friend ostream& operator<<(ostream& os, const $className& rhs)
    {
        return os;
    }
public: // attributes

private: // methods

private: // attributes
};

#endif
EOF

if [[ ! -e $cFileName ]]
then
#-----
# An example of a 'here' document.  It says cat (write) to $cFileName every
# thing after the marker EOF and before that same marker.  The '-' (<<-)
# says that leading tabs are allowed and will be stripped before writing
# the here document to the file
#-----
cat > $cFileName <<-EOF
/*=====
|
| File:          $cFileName
|
| Author:        $userName
|
| Date:          $(date +"%d %h %Y")
|
| Description:
|
+=====*/
#include <iostream>
#include "$className.hpp"

int main()
{

}
EOF
fi
#-----
# 'vi +11 fileName' means vi fileName and place the cursor on line 11.

```

```
# We want to place the cursor on the last line of fileName so that the user
# can immediately start editing the script, so first calculate the number
# of lines in the file: line=$(wc -l $fileName | awk '{print $1}'). Then
# exec vi: exec vi +$line $fileName. These two steps can be combined:
#-----
exec vi +$(grep -n class $fileName | sed 's/:.*//') $cFileName $hFileName
```

17.8 createMakefile

The following script creates a *makefile* with dependencies for all *c++* files in the current directory. It creates a makefile for an executable if one and only one of the files has a *main()* function. If none of the *c++* files have a *main()* function, on the other hand, the *makefile* for a *static* or *dynamic* library is created depending on a command switch.

```
#!/bin/bash
#=====
# File           : createMakefile
# Description     :
#
# Revision History:
#
#   18 Oct 2002 - marshallb (720-479-1511)
#       Initial Version.
#=====
# Function:      printHeader
# Description:   Print a makefile's header.
#=====
function printHeader
{
    local PROJECT=$1
    cat <<-EOH
        #=====
        # File           : makefile
        # Description    : Recursive makefile for directory $PROJECT
        #
        # $(date +"%d %h %Y") - $(grep ^$LOGNAME: /etc/passwd | cut -d: -f5)
        #=====
        EOH
    }

#=====
# Function:      printCCLine
# Description:   Print a CC line to the makefile. Here we are highlighting
#               some text using tput smso and un-highlighting by using
#               tput rmso.
#=====
function printCCLine
{
    echo "    @echo \$(shell tput smso)Creating \${@}\$(shell tput rmso) from \${<:"
```

```

dependencies \ $? are newer than \ $@"
    echo '    $(CXX) -c $(CFLAGS) $(ALL_INCLUDES) $<'
    echo
}

#=====
# Function:      printTargetBody
# Description:   Print the code to build a target.  Branch if a library
#               or an executable is being built
#=====
function printTargetBody
{
    local PROJECT=$1

    if [[ $PROJECT == *.so ]]
    then
        #-----
        # Body of a shared library
        #-----
        echo "\$(CXX) -shared -o $PROJECT \ $?"
    elif [[ $PROJECT == *.a ]]
    then
        #-----
        # Body of a static library
        #-----
        echo "\$(MAKELIB) $PROJECT \ $?"
    else
        #-----
        # Body of an executable
        #-----
        echo '\$(CXX) -o $@ $(CFLAGS) $(ALL_INCLUDES) $(ALL_LIBPATH) $(ALL_LIBS)
$^'
    fi
}

#=====
# Function      : printDependencies
# Description:   This function calculates a c (cpp, cpv) file's dependencies.
#               It depends on the ALL_INCLUDES macro in makefile.common.
#               If the g++ command fails, have the .o depend on the .c
#=====
function printDependencies
{
    local cFile=${1:-$(return)}
    local depFiles=$(g++ -MM $MY_INCLUDES $cFile)
    echo ${depFiles:-${cFile%.*}.o: $cFile}
}

#=====
# Function:      makeMakefile
# Description:   Creates a makefile to create an executable or a library
#               calls make in each of its subdirectories (and nothing
#               else)
#=====

```

```

function makeMakefile
{
    local PROJECT=$1
    #-----
    # create makefile
    #-----
    cat > makefile << EOF
$(printHeader $PROJECT)
SHELL = /usr/bin/bash

# master include
# include \$(NC3_SRC)/makefile.common

# my locals
include make.inc

# my masters
ALL_LIBS      = \$(MY_LIBS)
ALL_INCLUDES  = \$(MY_INCLUDES)
ALL_LIBPATH   = \$(MY_LIBPATH)

CFILES        = \$(wildcard *.cpp)
OBJS          = \$(addsuffix .o, \$(basename \$(wildcard *.cpp)))
VPATH         =
CFLAGS        += \$(MY_CFLAGS)
LDFLAGS       += \$(MY_LDFLAGS)

.PHONY: all mark sanity compile clean stable

%.o: %.cpp
    @echo \$(shell tput smso)Creating \${@}\$(shell tput rmso) from \${<:
dependencies \${?} are newer than \${@}
    \$(CXX) -c \$(CFLAGS) \$(ALL_INCLUDES) \${<

all: sanity compile $PROJECT

mark:
    @echo -----

sanity:
    @echo "-----"
    @echo "Operating with the following variables:"
    @echo "CFILES      = \$(CFILES)"
    @echo "OBJS        = \$(OBJS)"
    @echo "VPATH       = \$(VPATH)"
    @echo "CFLAGS      = \$(CFLAGS)"
    @echo "LDFLAGS     = \$(LDFLAGS)"
    @echo "ALL_INCLUDES = \$(ALL_INCLUDES)"
    @echo "ALL_LIBS    = \$(ALL_LIBS)"
    @echo "ALL_LIBPATH = \$(ALL_LIBPATH)"
    @echo "-----"

compile: \${OBJS} mark;

$PROJECT: \${OBJS}
    @echo "\$(shell tput smso)Building \${@}\$(shell tput rmso) from \${^} in
\$(CURDIR) "

```

```

$(printTargetBody $PROJECT)

clean:
    @echo "\$(shell tput smso)CLEANING\$(shell tput rmso) \$(OBJS) in
\$(CURDIR)"
    -@rm *.o
    -@rm $PROJECT
    -@rm \${NC3_LIB}/$PROJECT

stable:
    @echo "\$(shell tput smso)STABILIZING\$(shell tput rmso) \$(OBJS) in
\$(CURDIR)"
    \${MAKE} clean
    \${MAKE} all

$(for cFile in *.cpp
do
    echo $(printDependencies $cFile)
    printCCLine
done
)
EOF

if [[ ! -e make.inc ]]
then
    cat << EOF > make.inc
#=====
# File      : make.inc
# Description : Tailored flags for $PROJECT. This file is included by
#               makefile and will not be overwritten if by the makeMaker.
#               Put project specific details in this file.
#
# $(date +"%d %h %Y") - $(grep ^$LOGNAME: /etc/passwd | cut -d: -f5)
#=====
MY_INCLUDES = $MY_INCLUDES
MY_LIBS      = $MY_LIBS
MY_LIBPATH   = $MY_LIBPATH
MY_CFLAGS    = $([[ $PROJECT == *.so ]] && echo -fPIC)
MY_LDFLAGS   = $MY_LDFLAGS
EOF
else
    echo $(tput smso)Not overwriting make.inc$(tput rmso)
fi

echo "$(tput smso)makefile created in $(pwd) for $PROJECT$(tput rmso)"
}

#=====
# Function:    makeRecursiveMakefile
# Description: Create a recursive makefile, i.e. one which recursively
#               calls make in each of its subdirectories (and nothing
#               else)
#=====
function makeRecursiveMakefile
{
    local PROJECT=$(basename $(pwd))

```

```

    cat > makefile << EOF
$(printHeader $PROJECT)
SHELL = /usr/bin/bash

noParams:
    @echo Usage: make target
    @exit 1

%:
    @for f in `$(shell ls -ltr | grep '^d' | awk '{print \$\$NF}'); do \
        cd `\$\$f`; \
        `$(MAKE)` `\$@`; \
        cd ..; \
    done
EOF
echo "$(tput smso)makefile created in $(pwd) for $PROJECT$(tput rmso)"
}

#####
# Function:      makeExeMakefile
# Description: Create a makefile which generates an executable
#####
function makeExeMakefile
{
    fileName=$(egrep -l '[ \t]+main[ \t]*\(.*\)' *.c* | grep -v '//.*main'
2>/dev/null)
    makeMakefile `$(fileName%.*)`
}

#####
# Function:      makeLibMakefile
# Description: Create a makefile which generates a library
#####
function makeLibMakefile
{
    if [[ $libType == 'dynamic' ]]
    then
        makeMakefile "lib$(basename $(pwd)).so"
    else
        makeMakefile "lib$(basename $(pwd)).a"
    fi
}

#####
#
#                               MAIN
#
#####
# For directories in which no *.cpp files have a main() function,
# libType is set to dynamic, indicating that a shared library should be
# created. This can be overridden by using the -s option indicating
# a static library should be generated.
# This script takes options -l <libName> -L <libPathName> -I <include> as
# well. These options need be specified once, since they are written

```

```

# to the file make.inc, which this script doesn't overwrite.
#-----
libType=dynamic
#-----
# Read flags from make.inc if that file exists
#-----
if [[ -e make.inc ]]
then
    eval $(
        for f in $(grep MY make.inc )
        do
            echo $f
            done | xargs | sed 's/\([^=]\) \([^=]\)/\1; \2/g' | sed 's/ //g')
fi

while getopts :sc:d:l:L:I: option
do
    case $option in
        s) libType=static ;;
        c) MY_CFLAGS="$MY_CFLAGS $OPTARG" ;;
        d) MY_LDFLAGS="$MY_LDFLAGS $OPTARG" ;;
        l) MY_LIBS="$MY_LIBS -l$OPTARG" ;;
        L) MY_LIBPATH="$MY_LIBPATH -L$OPTARG" ;;
        I) MY_INCLUDES="$MY_INCLUDES -I$OPTARG" ;;
        *) echo -n "Usage: $(basename $0) -l <libName> -L <libdirName> "
           echo " -I <incdirName> -c <compileFlags> -d <linkerFlags>"
           exit ;;
    esac
done

#-----
# Strip the leading space
#-----
# MY_LIBS=${MY_LIBS# }
# MY_CFLAGS=${MY_CFLAGS# }
# MY_LDFLAGS=${MY_LDFLAGS# }
# MY_LIBPATH=${MY_LIBPATH# }
# MY_INCLUDES=${MY_INCLUDES# }

echo MY_LIBS = $MY_LIBS
echo MY_CFLAGS = $MY_CFLAGS
echo MY_LDFLAGS = $MY_LDFLAGS
echo MY_LIBPATH = $MY_LIBPATH
echo MY_INCLUDES = $MY_INCLUDES

#-----
# Create a recursive makefile (one which calls make for each of its
# subdirectories) if there are no cpp files in the current directory
#-----
if (( $(ls *.c *.cpp 2> /dev/null | wc -l) == 0 ))
then
    makeRecursiveMakefile
elif egrep '[ \t]+main[ \t]*\(.*)' *.c* | grep -v '//.*main'> /dev/null 2>&1
then
    #-----
    # There are cpp files in the current directory and one of them has
    # a main() function, so build a makefile for an executable

```

```
#-----  
makeExeMakefile  
else  
#-----  
# None of the cpp files has a main() function.  So build a makefile  
# for a library  
#-----  
makeLibMakefile  
fi
```

18 Appendix A Bourne Shell Variables

Bash uses certain shell variables in the same way as the Bourne shell. In some cases, Bash assigns a default value to the variable.

CDPATH

A colon-separated list of directories used as a search path for the `cd` builtin command.

HOME

The current user's home directory; the default for the `cd` builtin command. The value of this variable is also used by tilde expansion

IFS

A list of characters that separate fields; used when the shell splits words as part of expansion.

MAIL

If this parameter is set to a filename and the `MAILPATH` variable is not set, Bash informs the user of the arrival of mail in the specified file.

MAILPATH

A colon-separated list of filenames which the shell periodically checks for new mail. Each list entry can specify the message that is printed when new mail arrives in the mail file by separating the file name from the message with a ``?'`. When used in the text of the message, `$_` expands to the name of the current mail file.

OPTARG

The value of the last option argument processed by the `getopts` builtin.

OPTIND

The index of the last option argument processed by the `getopts` builtin.

PATH

A colon-separated list of directories in which the shell looks for commands.

PS1

The primary prompt string. The default value is ``s-\\$ '`.

PS2

The secondary prompt string. The default value is ``> '`.

19 Appendix B Bash Variables

These variables are set or used by Bash, but other shells do not normally treat them specially.

BASH

The full pathname used to execute the current instance of Bash.

BASH_ENV

If this variable is set when Bash is invoked to execute a shell script, its value is expanded and used as the name of a startup file to read before executing the script.

BASH_VERSION

The version number of the current instance of Bash.

BASH_VERSINFO

A readonly array variable whose members hold version information for this instance of Bash. The values assigned to the array members are as follows:

BASH_VERSINFO[0]

The major version number (the *release*).

BASH_VERSINFO[1]

The minor version number (the *version*).

BASH_VERSINFO[2]

The patch level.

BASH_VERSINFO[3]

The build version.

BASH_VERSINFO[4]

The release status (e.g., *beta1*).

BASH_VERSINFO[5]

The value of MACHTYPE.

COLUMNS

Used by the select builtin command to determine the terminal width when printing selection lists. Automatically set upon receipt of a SIGWINCH.

COMP_CWORD

An index into \${COMP_WORDS} of the word containing the current cursor position. This variable is available only in shell functions invoked by the programmable completion facilities.

COMP_LINE

The current command line. This variable is available only in shell functions and external commands invoked by the programmable completion facilities.

COMP_POINT

The index of the current cursor position relative to the beginning of the current command. If the current cursor position is at the end of the current command, the value of this variable is equal to \${#COMP_LINE}. This variable is available only in shell functions and external commands invoked by the programmable completion facilities.

COMP_WORDS

An array variable consisting of the individual words in the current command line. This variable is available only in shell functions invoked by the programmable completion facilities.

COMP_REPLY

An array variable from which Bash reads the possible completions generated by a shell function invoked by the programmable completion facility.

DIRSTACK

An array variable containing the current contents of the directory stack. Directories appear in the stack in the order they are displayed by the `dirs` builtin. Assigning to members of this array variable may be used to modify directories already in the stack, but the `pushd` and `popd` builtins must be used to add and remove directories. Assignment to this variable will not change the current directory. If `DIRSTACK` is unset, it loses its special properties, even if it is subsequently reset.

EUID

The numeric effective user id of the current user. This variable is readonly.

FCEDIT

The editor used as a default by the `-e` option to the `fc` builtin command.

FIGIGNORE

A colon-separated list of suffixes to ignore when performing filename completion. A file name whose suffix matches one of the entries in `FIGIGNORE` is excluded from the list of matched file names. A sample value is `.o:~'`

FUNCNAME

The name of any currently-executing shell function. This variable exists only when a shell function is executing. Assignments to `FUNCNAME` have no effect and return an error status. If `FUNCNAME` is unset, it loses its special properties, even if it is subsequently reset.

GLOBIGNORE

A colon-separated list of patterns defining the set of filenames to be ignored by filename expansion. If a filename matched by a filename expansion pattern also matches one of the patterns in `GLOBIGNORE`, it is removed from the list of matches.

GROUPS

An array variable containing the list of groups of which the current user is a member. Assignments to `GROUPS` have no effect and return an error status. If `GROUPS` is unset, it loses its special properties, even if it is subsequently reset.

histchars

Up to three characters which control history expansion, quick substitution, and tokenization. The first character is the *history expansion* character, that is, the character which signifies the start of a history expansion, normally `!`. The second character is the character which signifies 'quick substitution' when seen as the first character on a line, normally `^`. The optional third character is the character which indicates that the remainder of the line is a comment when found as the first character of a word, usually `#`. The history comment character causes history substitution to be skipped for the remaining words on the line. It does not necessarily cause the shell parser to treat the rest of the line as a comment.

HISTCMD

The history number, or index in the history list, of the current command. If HISTCMD is unset, it loses its special properties, even if it is subsequently reset.

HISTCONTROL

A value of *'ignorespace'* means to not enter lines which begin with a space or tab into the history list. A value of *'ignoredups'* means to not enter lines which match the last entered line. A value of *'ignoreboth'* combines the two options. Unset, or set to any other value than those above, means to save all lines on the history list. The second and subsequent lines of a multi-line compound command are not tested, and are added to the history regardless of the value of HISTCONTROL.

HISTFILE

The name of the file to which the command history is saved. The default value is *'~/.bash_history'*.

HISTFILESIZE

The maximum number of lines contained in the history file. When this variable is assigned a value, the history file is truncated, if necessary, to contain no more than that number of lines. The history file is also truncated to this size after writing it when an interactive shell exits. The default value is 500.

HISTIGNORE

A colon-separated list of patterns used to decide which command lines should be saved on the history list. Each pattern is anchored at the beginning of the line and must match the complete line (no implicit *'*'* is appended). Each pattern is tested against the line after the checks specified by HISTCONTROL are applied. In addition to the normal shell pattern matching characters, *'&'* matches the previous history line. *'&'* may be escaped using a backslash; the backslash is removed before attempting a match. The second and subsequent lines of a multi-line compound command are not tested, and are added to the history regardless of the value of HISTIGNORE.

HISTIGNORE subsumes the function of HISTCONTROL. A pattern of *'&'* is identical to *ignoredups*, and a pattern of *'[]*'* is identical to *ignorespace*. Combining these two patterns, separating them with a colon, provides the functionality of *ignoreboth*.

HISTSIZE

The maximum number of commands to remember on the history list. The default value is 500.

HOSTFILE

Contains the name of a file in the same format as *'/etc/hosts'* that should be read when the shell needs to complete a hostname. The list of possible hostname completions may be changed while the shell is running; the next time hostname completion is attempted after the value is changed, Bash adds the contents of the new file to the existing list. If HOSTFILE is set, but has no value, Bash attempts to read *'/etc/hosts'* to obtain the list of possible hostname completions. When HOSTFILE is unset, the hostname list is cleared.

HOSTNAME

The name of the current host.

HOSTTYPE

A string describing the machine Bash is running on.

IGNOREEOF

- Controls the action of the shell on receipt of an EOF character as the sole input. If set, the value denotes the number of consecutive EOF characters that can be read as the first character on an input line before the shell will exit. If the variable exists but does not have a numeric value (or has no value) then the default is 10. If the variable does not exist, then EOF signifies the end of input to the shell. This is only in effect for interactive shells.
- INPUTRC**
The name of the Readline initialization file, overriding the default of `~/inputrc`.
- LANG**
Used to determine the locale category for any category not specifically selected with a variable starting with `LC_`.
- LC_ALL**
This variable overrides the value of `LANG` and any other `LC_` variable specifying a locale category.
- LC_COLLATE**
This variable determines the collation order used when sorting the results of filename expansion, and determines the behavior of range expressions, equivalence classes, and collating sequences within filename expansion and pattern matching.
- LC_CTYPE**
This variable determines the interpretation of characters and the behavior of character classes within filename expansion and pattern matching.
- LC_MESSAGES**
This variable determines the locale used to translate double-quoted strings preceded by a ``$'`.
- LC_NUMERIC**
This variable determines the locale category used for number formatting.
- LINENO**
The line number in the script or shell function currently executing.
- LINES**
Used by the `select` builtin command to determine the column length for printing selection lists. Automatically set upon receipt of a `SIGWINCH`.
- MACHTYPE**
A string that fully describes the system type on which Bash is executing, in the standard GNU *cpu-company-system* format.
- MAILCHECK**
How often (in seconds) that the shell should check for mail in the files specified in the `MAILPATH` or `MAIL` variables. The default is 60 seconds. When it is time to check for mail, the shell does so before displaying the primary prompt. If this variable is unset, or set to a value that is not a number greater than or equal to zero, the shell disables mail checking.
- OLDPWD**
The previous working directory as set by the `cd` builtin.
- OPTERR**
If set to the value 1, Bash displays error messages generated by the `getopts` builtin command.
- OSTYPE**
A string describing the operating system Bash is running on.
- PIPESTATUS**

An array variable containing a list of exit status values from the processes in the most-recently-executed foreground pipeline (which may contain only a single command).

POSIXLY_CORRECT

If this variable is in the environment when bash starts, the shell enters POSIX mode before reading the startup files, as if the `--posix` invocation option had been supplied. If it is set while the shell is running, bash enables POSIX mode, as if the command `set -o posix` had been executed.

PPID

The process ID of the shell's parent process. This variable is readonly.

PROMPT_COMMAND

If set, the value is interpreted as a command to execute before the printing of each primary prompt (\$PS1).

PS3

The value of this variable is used as the prompt for the select command. If this variable is not set, the select command prompts with ``#? '`

PS4

The value is the prompt printed before the command line is echoed when the ``-x'` option is set. The first character of PS4 is replicated multiple times, as necessary, to indicate multiple levels of indirection. The default is ``+ '`.

PWD

The current working directory as set by the cd builtin.

RANDOM

Each time this parameter is referenced, a random integer between 0 and 32767 is generated. Assigning a value to this variable seeds the random number generator.

REPLY

The default variable for the read builtin.

SECONDS

This variable expands to the number of seconds since the shell was started. Assignment to this variable resets the count to the value assigned, and the expanded value becomes the value assigned plus the number of seconds since the assignment.

SHELLOPTS

A colon-separated list of enabled shell options. Each word in the list is a valid argument for the ``-o'` option to the set builtin command. The options appearing in SHELLOPTS are those reported as ``on'` by ``set -o'`. If this variable is in the environment when Bash starts up, each shell option in the list will be enabled before reading any startup files. This variable is readonly.

SHLVL

Incremented by one each time a new instance of Bash is started. This is intended to be a count of how deeply your Bash shells are nested.

TIMEFORMAT

The value of this parameter is used as a format string specifying how the timing information for pipelines prefixed with the time reserved word should be displayed. The ``%'` character introduces an escape sequence that is expanded to a time value or other information. The escape sequences and their meanings are as follows; the braces denote optional portions.

%%

A literal ``%'`.

`%[p][l]R`

The elapsed time in seconds.

`%[p][l]U`

The number of CPU seconds spent in user mode.

`%[p][l]S`

The number of CPU seconds spent in system mode.

`%P`

The CPU percentage, computed as $(\%U + \%S) / \%R$.

The optional *p* is a digit specifying the precision, the number of fractional digits after a decimal point. A value of 0 causes no decimal point or fraction to be output. At most three places after the decimal point may be specified; values of *p* greater than 3 are changed to 3. If *p* is not specified, the value 3 is used.

The optional *l* specifies a longer format, including minutes, of the form *MMmSS.FFs*. The value of *p* determines whether or not the fraction is included.

If this variable is not set, Bash acts as if it had the value

`$'\nreal\t%3lR\nuser\t%3lU\nsys\t%3lS'`. If the value is null, no timing information is displayed. A trailing newline is added when the format string is displayed.

TMOUT

If set to a value greater than zero, the value is interpreted as the number of seconds to wait for input after issuing the primary prompt when the shell is interactive. Bash terminates after that number of seconds if input does not arrive.

UID

The numeric real user id of the current user. This variable is readonly.

20 Appendix C Command-line processing

When a user types a command at the prompt, Bash processes the command in the following order:

1. The command line is split into tokens delimited by the fixed set of meta-characters, SPACE, TAB, NEWLINE, ;, (,), <, >, | and &. Tokens include *words*, *keywords*, *I/O redirectors*, and *semicolons*
2. Checks if the first token of each command is an *opening* keyword (such as *if* and other control-structure openers, *function*, {, or () with no quotes or backslashes. If it is, then the command is a *compound command* and the shell starts at step 1 again. If the keyword isn't an opening keyword, a syntax error is reported and processing stops.
3. Checks the first word to see if it is an alias. If so, restart at step 1.
4. Perform *brace expansion*
5. Substitute \$HOME for '~' if it is at the beginning of a word
6. Perform variable name expansion
7. Perform command substitution
8. Evaluate arithmetic expressions
9. Retokenize the command string obtained by applying steps 6, 7, 8, using \$IFS as the delimiter
10. Perform *pathname (wildcard) expansion*
11. Use the first word as a command. Default order for command lookup is: functions, built-ins, scripts/executables.
12. Set up I/O redirection
13. Run the command

21 Appendix D FAQs

21.1 How do I keep the *read* builtin from skipping leading whitespace?

Set IFS (the Internal Field Separator) to the empty string. Thus the following code faithfully copies a file:

```
#!/bin/bash
#=====
# File           : copyTest
# Description    :
#
# Revision History:
#
#   13 Feb 2003 - bmarshal
#       Initial Version.
#=====
srcFile=${1:?Usage: $0 <srcFile> <destFile>}
desFile=${2:?Usage: $0 <srcFile> <destFile>}

OLD_IFS=$IFS
IFS=''

while read line
do
    echo $line >> $desFile
done < $srcFile

IFS=$OLD_IFS
```

21.2 How do I read from the keyboard or write to the terminal when STDIN or STDOUT is redirected?

read from /dev/tty

\$ read input < /dev/tty

\$ echo string > /dev/tty

21.3 How do I strip duplicate whitespace from a string?

The preferred idiom is:

\$ line=\$(echo \$line)

21.4 Can a variable have a default value?

Yes, for the variable *var*, `${var:-defaultValue}` has the value `$var` if it is defined and not null or *defaultValue* otherwise.

21.5 How can we reset `$*` efficiently?

The `*` variable is assigned to in at least three ways: it contains the list of command line arguments in global scope, the list of function parameters within the body of a function, but can also be set using the `set` command. The idiom:

```
$ set –          # double dash
```

resets the `*` variable. This is more efficient than:

```
$ shift $#       # recall that $# represents the number of words in $*
```

22 Appendix E Regular Expression Synopsis

Operator	Prec.	Function
.		Match any ascii character except the new line
[]	2	Character class (one of the set), – indicates range: a-z
[^]	2	Complement character class (not one of the set)
\	1	Quote the next metacharacter
^	5	Anchor operator (beginning of line)
\$	5	Anchor operator (end of line)
\<		Beginning of word anchor, words are bounded by white space, beginning/end of line, and punctuation
\>		End of word anchor, words are bounded by white space, beginning/end of line, and punctuation
\{n\}	4	Quantifier, number of occurrences, exactly n
\{n,\}	4	Quantifier, numerical range of n or more
\{n,m\}	4	Quantifier, numerical range where n <= m < 256
{ }	4	Same as preceding 3 lines for Perl only
?	4	Quantifier, zero or one, equivalent to \{0,1\}
+	4	Quantifier, one or more, equivalent to \{1,\}
*	4	Quantifier, zero or more, equivalent to \{0,\}
\(\)	3	Grouping operator
()	3	Grouping operator
\digit	3	Value of previously matched group occurrence, up to \9
	6	Alternation operator
&		Value of the matched regular expression, in substitution

Utility Regular Expression Chart

RE	ed	grep	egrep	sed	awk	perl
.	✓	✓	✓	✓	✓	✓
[]	✓	✓	✓	✓	✓	✓
\	✓	✓	✓	✓	✓	✓
^	✓	✓	✓	✓	✓	✓
\$	✓	✓	✓	✓	✓	✓
\<		✓		✓		
\>		✓		✓		
\{ \}	✓	✓		✓		
{ }						✓
?			✓		✓	✓
+			✓		✓	✓
*	✓	✓	✓	✓	✓	✓
\(\)	✓	✓		✓		
()			✓		✓	✓
\digit	✓	✓		✓		✓
\$digit						✓
			✓		✓	✓
&	✓			✓	✓	